



Designing a Web Application using the Twitter streaming API as a Tracking Tool for Weather Updates

By

Adrian C. Ablazo

School of Arts and Sciences
Ateneo de Davao University

February 2019

Designing a Web Application using the Twitter streaming API as a Tracking Tool for Weather Updates

An Independent Study
Presented to the
Graduate Faculty of the
Computer Studies cluster
Ateneo de Davao University

In Partial Fulfillment
of the Requirements for the Degree
Master of Science in Information Technology

By
Adrian C. Ablazo

SCHOOL OF ARTS AND SCIENCES
ATENEO DE DAVAO UNIVERSITY

February 2019

Ateneo de Davao University

COMPUTER STUDIES CLUSTER
SCHOOL OF ARTS AND SCIENCES
P.O. Box 8016 Davao City
Philippines



Recommendation for Oral Defense

*In partial fulfillment of the requirements for the degree of **MASTER OF SCIENCE IN INFORMATION TECHNOLOGY**, this **GRADUATE INDEPENDENT STUDY** entitled:*

DESIGNING A WEB APPLICATION USING THE TWITTER STREAMING API AS A TRACKING TOOL FOR WEATHER UPDATES

has been prepared and submitted by ADRIAN C. ABLAZO and is recommended for ORAL FINAL DEFENSE.

GRACE S. TACADAO

Mentor

Ateneo de Davao University

COMPUTER STUDIES CLUSTER
SCHOOL OF ARTS AND SCIENCES
P.O. Box 8016 Davao City
Philippines



Recommendation for Acceptance

*The faculty of Computer Studies Cluster of the Ateneo de Davao University
accepts the Graduate Independent Study entitled:*

DESIGNING A WEB APPLICATION USING THE TWITTER STREAMING API AS A TRACKING TOOL FOR WEATHER UPDATES

*which has been prepared and submitted by ADRIAN C. ABLAZO in partial
fulfillment of the requirements for the degree Master of Science in Information
Technology.*

FR. RAMON PRUDENCIO S. TOLEDO, S.J., M.Sc.
Panelist

FR. CARLOS G. CENZON, Jr., S.J., Ph.D.
Panelist

MICHELLE P. BANAWAN, Ph.D.
Chair Panelist

Acknowledgments

I would like to thank God for granting me the patience and perseverance to finish this paper.

I would like to thank my mother, *Reggie*, my father, *Romy* and my sister, *Kym*, for their constant words of encouragement and support through the entirety of my academic life.

I would like to thank *Ma'am Grace*, my adviser, for imparting her knowledge on Python and Data Science, as well as giving helpful comments on my paper.

I would like to thank my defense panelists, *Ma'am Michelle*, for giving helpful insights on my methodologies and for always preaching to us all, *Data is Beautiful!* I would also like to thank, *Father Denny* and *Father Charlie*, for their valuable inputs and recommendations.

I would like to thank *Raul* and *Pating*, my fellow junior faculty in the CS cluster, who served as my inspirations in finishing my paper.

I would like to thank *Sir Rey*, *Ma'am Rubye*, *Ate Aneth*, and the rest of the CS faculty and staff for their support and assistance in my study.

Lastly, I would also like to thank my former students, who participated in the annotation process of my data, and taught me that life is great even without annotations.

Table of Contents

LIST OF FIGURES	VIII
LIST OF TABLES	IX
CHAPTER 1 INTRODUCTION	1
1.1 BACKGROUND OF THE PROBLEM	1
1.2 STATEMENT OF THE PROBLEM	2
1.3 OBJECTIVES OF THE STUDY.....	2
1.4 SIGNIFICANCE OF THE STUDY	2
1.5 SCOPE AND LIMITATIONS OF THE STUDY	2
1.6 DEFINITION OF TERMS	3
CHAPTER 2 REVIEW OF RELATED LITERATURE	4
2.1 TWITTER AS A COMMUNICATION TOOL IN EMERGENCIES.....	4
2.2 THE CLASSIFICATION OF TWEET CORPUS IN EMERGENCIES	4
2.3 CORPUS SENTIMENT ANALYSIS.....	5
2.3.1 <i>Naïve Bayes</i>	6
2.3.2 <i>N-Grams</i>	6
2.4 TWITTER API.....	7
2.4.1 <i>Web Applications which uses Twitter's streaming API</i>	7
2.4.2 <i>Tweepy</i>	7
2.5 DATA VISUALIZATION.....	8
2.6 DJANGO WEB FRAMEWORK.....	8
2.6.1 <i>Python Django for Scientific Web Applications</i>	9
2.7 DATA-DRIVEN DOCUMENTS (D3.JS).....	9
2.7.1 <i>Circle Packing</i>	9
CHAPTER 3 THEORETICAL FRAMEWORK.....	11
3.1 GATHER THE STORM-RELATED TWEETS.....	11
3.2 MANUAL CLASSIFICATION OF TWEETS.....	11
3.3 AUTOMATIC CLASSIFICATION OF TWEETS.....	12
3.4 EVALUATION	12
CHAPTER 4 RESEARCH DESIGN AND METHODOLOGY.....	13
4.1 CONCEPTUAL FRAMEWORK	13
4.2 METHODOLOGY	13
4.2.1 <i>Gather Tweet Dataset</i>	13
4.2.2 <i>Cleaning of Tweet Dataset</i>	15
4.2.3 <i>Manual Classification of Tweets</i>	18
4.2.4 <i>Data Split</i>	21
4.2.5 <i>Feature Extraction</i>	22
4.2.6 <i>Creating the automatic classifiers</i>	26
4.2.7 <i>Evaluation</i>	27
CHAPTER 5 RESULTS AND DISCUSSION	29
5.1 COMPARISON OF THE EVALUATION RESULTS.....	29

5.2	IMPLEMENTING THE WEB APPLICATION	36
5.2.1	<i>Exporting result data</i>	37
5.2.2	<i>Creating the JSON file</i>	39
5.2.3	<i>Implementing the Visualization of Results in Django</i>	44
CHAPTER 6	CONCLUSIONS AND RECOMMENDATIONS.....	52
6.1	CONCLUSION	52
6.2	RECOMMENDATIONS	53
REFERENCES		54
APPENDICES		56
APPENDIX A –	IMPORTED PYTHON LIBRARIES.....	56
APPENDIX B –	SCRIPT TO UNDERSAMPLE 6-CLASS DATASET.....	57
APPENDIX C –	SCRIPT TO DATA SPLIT FOR THE 2-CLASS AND 6-CLASS DATASET.....	58
APPENDIX D –	GRAPHICAL CONFUSION MATRIX PLOTTER SCRIPT	59
APPENDIX E –	SCRIPT OF THE BNB CLASSIFIER, EVALUATOR AND CONFUSION MATRIX PLOTTER FOR THE 6-CLASS DATASET.....	60
APPENDIX F –	SCRIPT OF THE MNB CLASSIFIER, EVALUATOR AND CONFUSION MATRIX PLOTTER FOR 6-CLASS DATASET SCRIPT	61
APPENDIX G –	SCRIPT OF THE BNB CLASSIFIER, EVALUATOR AND CONFUSION MATRIX PLOTTER FOR THE 2-CLASS DATASET.....	62
APPENDIX H –	SCRIPT OF THE MNB CLASSIFIER, EVALUATOR AND CONFUSION MATRIX PLOTTER FOR THE 2-CLASS DATASET.....	63
APPENDIX I –	SCRIPT OF THE AUC-ROC CURVE AND SCORE COMPARISON BETWEEN MNB CLASSIFIERS WITH N-GRAM (1,4) AND (1,6) FEATURES	64
APPENDIX J –	SCRIPT TO EXPORT PREDICTED RESULTS FOR 2-CLASS DATASET	65
APPENDIX K –	SCRIPT TO EXPORT PREDICTED RESULTS FOR 6-CLASS DATASET.....	66
APPENDIX L –	SCRIPT TO GENERATE N-GRAM COLUMNS (1-6) FOR 2-CLASS DATASET.....	67
APPENDIX M -	SCRIPT TO GENERATE N-GRAM COLUMNS (1-4) FOR 6-CLASS DATASET	70
APPENDIX N –	PYTHON DJANGO’S SETTINGS.PY.....	72
APPENDIX O –	PYTHON DJANGO’S URLS.PY	73
APPENDIX P –	PYTHON DJANGO’S FORMS.PY	74
APPENDIX Q –	PYTHON DJANGO’S VIEWS.PY	75
APPENDIX R –	BASE.HTML THE BASE TEMPLATE FOR ALL WEB PAGES	79
APPENDIX S –	CIRCLE_PACK.HTML, THE TEMPLATE WHICH IMPLEMENTS THE D3.JS CIRCLE PACKING CODES	80
APPENDIX T –	LIST OF STOP WORDS	82

LIST OF FIGURES

FIGURE 1: DJANGO'S MODEL-VIEW-TEMPLATE ARCHITECTURE	8
FIGURE 2: CIRCLE PACKING	10
FIGURE 3: THE CONCEPTUAL FRAMEWORK	13
FIGURE 4: SCRIPT FOR COLLECTING NONAPH TWEETS USING TWEETPY	14
FIGURE 5: PYTHON SCRIPT USED TO REMOVE NON-ALPHANUMERIC SYMBOLS FROM THE TWEET DATASET	16
FIGURE 6: PYTHON SCRIPT USED TO REMOVE URLS, MENTIONS, AND HASHTAGS FROM THE TWEET DATASET	16
FIGURE 7: PYTHON SCRIPT USED TO REPLACE BLANK CELLS IN THE TWEET DATASET WITH 'NONE'	17
FIGURE 8: A PYTHON SCRIPT TO REMOVE DUPLICATES AND RETWEETS FROM THE TWEET DATASET	17
FIGURE 9: VIEW OF SAMPLE TWEET SURVEY WITH SIX CATEGORIES IN GOOGLE FORMS	18
FIGURE 10: SCRIPT TO REMOVE ROWS THAT DIDN'T HAVE LABEL AGREEMENTS	19
FIGURE 11: SNIPPET OF THE DATASET WITH THE CORRESPONDING DEFINED_CLASS VALUES	20
FIGURE 12: SCRIPT TO RANDOMLY UNDER SAMPLE EACH CLASS	21
FIGURE 13: SCRIPT TO SPLIT AND ASSIGN THE 6-CLASS DATASET INTO ITS TRAIN AND TEST DATASETS	22
FIGURE 14: SCRIPT TO SPLIT AND ASSIGN THE 2-CLASS DATASET INTO ITS TRAIN AND TEST DATASETS	22
FIGURE 15: FEATURES GENERATED BY FIT_TRANSFORM FOR THE FIRST TWO ROWS OF THE TRAINING DATA SET.	24
FIGURE 16: THE TERM-DOCUMENT MATRIX OF THE FIRST TWO ROWS OF THE TRAINING DATASET	24
FIGURE 17: THE TF-IDF REPRESENTATION OF THE FIRST TWO ROWS OF THE TRAINING DATASET.	25
FIGURE 18: SCRIPT TO INITIALIZE THE PIPELINE, WITH VARYING N-GRAM FEATURES	26
FIGURE 19: SCRIPT TO TRAIN THE DATASETS AND GENERATE A PREDICTION	26
FIGURE 20: SCRIPT TO DETERMINE ACCURACY SCORE AND CLASSIFICATION REPORT.	27
FIGURE 21: CONFUSION MATRICES OF BNB AND MNB AT RANGE (1,4) FOR THE 6-CLASS DATASET	33
FIGURE 22: CONFUSION MATRICES OF BNB AND MNB AT RANGE (1,4) FOR THE 2-CLASS DATASET	34
FIGURE 23: CONFUSION MATRICES OF BNB AND MNB AT RANGE (1,6) FOR THE 2-CLASS DATASET	34
FIGURE 24: AUC-ROC CURVE AND AUC SCORE FOR BOTH MNB MODELS	36
FIGURE 25: DATA FLOW FROM CLASSIFICATION TO VISUALIZATION	37
FIGURE 26: SCRIPT TO EXPORT RESULT N-GRAM RANGE (1,4) DATA	37
FIGURE 27: SCRIPT TO EXPORT AND RELABEL RESULT N-GRAM RANGE (1,6) DATA	38
FIGURE 28: SCRIPT TO REMOVE DUPLICATES IN THE INTERSECTION OF DATA FRAMES	39
FIGURE 29: SCRIPT TO OMIT ANY CONFLICTING PREDICTED AND MANUALLY CLASSIFIED LABELS	39
FIGURE 30: SCRIPT TO SAVE THE DATA FRAME INTO A CSV FILE	39
FIGURE 31: SCRIPT TO IDENTIFY 1-GRAM TO 6-GRAM TERMS	40
FIGURE 32: HIERARCHY OF CLASSES AND CORRESPONDING CHILD CLASSES FOR THE 2-CLASS DATASET	42
FIGURE 33: HIERARCHY OF CLASSES AND CORRESPONDING CHILD CLASSES FOR THE 6-CLASS DATASET	43

LIST OF TABLES

TABLE 1: LANDIS AND KOCH’S FLEISS KAPPA INTERPRETATION	20
TABLE 2: SUMMARY OF ACCURACY SCORES AND CLASSIFICATION RESULTS FOR BNB IN 6-CLASS DATASET	30
TABLE 3: SUMMARY OF ACCURACY SCORES AND CLASSIFICATION RESULTS FOR MNB IN 6-CLASS DATASET	31
TABLE 4: SUMMARY OF ACCURACY SCORES AND CLASSIFICATION RESULTS FOR BNB IN 2-CLASS DATASET	32
TABLE 5: SUMMARY OF ACCURACY SCORES AND CLASSIFICATION RESULTS FOR MNB IN 2-CLASS DATASET	32
TABLE 6: N-GRAM RANGE (1,6) FOR TWEET TEXT	41

ABSTRACT

With the numerous storms and typhoons that have plagued the Philippines, social media websites such as *Twitter* have proven to be of aid to people in the communication and dissemination of information between the local government units and citizenry. The national government implements and uses a unified *hashtag* convention to easily track and monitor any news or information regarding these disasters. However, not all users utilized these *hashtags* to spread newsworthy tweets. Some of them sent tweet messages that were not meaningful for the intended purpose. Thus, this study aims to determine the bases for the validity of news posted in social networking sites, such as Twitter, to use data mining techniques on *Twitter's* streaming API to gather past tweet data and to classify these tweet data in an automated implementation of classifying tweets with storm-related news.

Keywords: Twitter, storms, hashtag, streaming API, data mining

CHAPTER 1

INTRODUCTION

1.1 Background of the Problem

Twitter is a free micro-blogging service with over a million users worldwide. It allows users to post and exchange tweet messages with up to 280 characters. Twitter is used through a wide variety of users. As of 2010, about 46% of active users correspond to mobile users (Williams, 2010). However, not all tweets are informative. Most Twitter users post about themselves (Naaman, Boase, & Lai, 2010). Although most tweets are considered unimportant, others have interesting and relevant information, and most of these relevant tweets are associated with hashtags. A hashtag is a word or phrase preceded by the # symbol that classifies or categorizes the accompanying text (Hashtag definition by Merriam-Webster, 2015).

Hashtags are also indicators of a tweet's meaning, topic, or, its intended audience (Efron, 2010). Due to its categorical aspect, it is used by news agencies as a flag for them to gather data related to certain situations from the masses, such as the arrival of a typhoon or storm. However, not even all tweets with corresponding hashtags about the storm's arrival are deemed informatively valid. Some tweets spread rumors, false news, and even irrelevant personal information. Thus, there is a need to know what tweet is valid or relevant during storm arrivals.

To do this, there is a need to analyze a tweet's corpus for relevance. There have been studies which analyze tweet corpus in times of emergencies, storms, and earthquakes. These studies do tweet data mining, using Twitter's streaming API, followed by classification and labeling of tweets. Though relevant tweets were classified appropriately for their content, these were done manually, using crowdsourcing services. Crowdsourcing involves a lot of respondents to get the job done, usually with the help of online crowdsourcing services. There had been studies which use crowdsourcing to manually label datasets to implement an automated method of classifying data. Thus, there is also a need to do an automated method of classifying tweet data.

There had been no attempt of implementing an automated method of classifying tweet data according to its content in the context of disaster communications in the Philippines through a web visualization tool. Thus, the

research proponent wishes to pursue this study to implement an automated method of classifying Tagalog and English tweets for their validity and relevance.

1.2 Statement of the Problem

This study is conducted to investigate Twitter's streaming API and its capabilities to track tweets during storm-related events. Also, this study seeks to answer the following questions:

- How can Twitter's streaming API gather or data mine tweets?
- What are the necessary tools needed to analyze the tweet corpora?
- What are the factors and criteria by which a tweet is considered valid?
- What are the automated methods that can be used for tweet classification?
- What are the methods of tweet post-processing?

1.3 Objectives of the Study

The main objective of this study is to investigate Twitter's streaming API and to determine its capabilities to track tweets during storm-related events. The study also wishes to accomplish the following objectives:

- To explain how Twitter's streaming API gather or data mine tweets.
- To identify the necessary tools needed to analyze the tweet corpora.
- To identify the factors and criteria by which a tweet is considered valid.
- To identify and implement an automated method of tweet classification.
- To identify methods of tweet post-processing.

1.4 Significance of the Study

The output of this study can be used by media practitioners and concerned citizens in ascertaining what valid storm-tweets are. The output would also allow citizens within reach of an ongoing storm to have a better grasp on the conditions of areas affected by natural calamities caused by storms. This study may also serve as one of the few types of research on the area of data mining in microblogging sites in the Philippine context.

1.5 Scope and Limitations of the Study

The study is conducted to determine how storm-related tweets can be ascertained for their validity in the Philippines. The study will only focus on past storm-related tweet data in the Philippines. It will only focus on tweets with the Philippine unified storm hashtag convention. The study will not focus on other natural calamities, such as massive earthquakes or fires. The tweet

data to be used in this study will only be the tweets from public accounts, using the Filipino and English language. This study covers the use of Twitter's streaming API for data mining text-data in tweets, and the both automated and manual classification of tweets.

1.6 Definition of Terms

- Hashtag - is a word or phrase preceded by the # symbol that classifies or categorizes the accompanying text (Hashtag definition by Merriam-Webster, 2015).
- Corpus (pl. Corpora) - a collection of written texts, especially the entire works of an author or a body of writing on a particular subject.
- Trending topic – A list pane found on either the right or left-hand side of the Twitter page, which shows the current most popular and talked about topics in the site, in real time.
- Mention - a feature which *tags* users, usernames, or a user's screen name in social networking sites, by preceding the said information with the @ symbol. This was originally a feature from Twitter but has recently been implemented in other SNSs such as Facebook.
- Retweet (abbrv. *RT*) - a feature in twitter which allows users to repost another user's tweet, given that it is publicly available.
- Emoticon - a combination of the words 'emotion' and 'icon'. This is a group of selected symbols that are stacked together to form a facial expression, used to help users better express themselves in digital communications.

CHAPTER 2

REVIEW OF RELATED LITERATURE

2.1 Twitter as a communication tool in emergencies

Twitter is an online microblogging service which is used as a communication tool by users all around the world. This service is used by many users for personal communication, but one of the most relevant uses of the said service is to spread information about emergencies or even events, which entice mass convergence. It had been used as a tool to spread information about emergencies about floods, storms, and earthquakes. Due to this fast and free method of online communication, Twitter had been used in previous studies about human behavior during natural calamities (De Leon, Estuar, Lee, & Ybañez, 2013). But it was also discovered that the tweets that users send in the dire times of storm arrivals or earthquakes are not entirely relevant and could contain rumors and false news (Castillo, Mendoza, & Poblete, 2010). To this end, there had been studies that provide a basis to identify and classify tweets with situational awareness in times of disasters (Vieweg, 2012).

2.2 The classification of Tweet corpus in emergencies

Tweets written in the wake of a storm disaster are varied and free-flowing. Therefore, there is a need to classify each tweet's corpus – to determine its relevance. A relevant tweet in the context of mass emergencies is a tweet, which provides *situational awareness*. Tweets considered to have situational awareness must provide tactical, actionable information that can aid people in making decisions, advise others on how to obtain specific information from various sources, or offer immediate post-impact help to those affected by the mass emergency. A study presented a total of 34 specific types of information that contributes to situational awareness, these were: Advice – Information Space, Animal Management, Caution, Evacuation, Fatality, Feeding/Hydration, General Population Information, Immediate Recovery, Injury, Medical Attention, Missing, Offer of Help, Preparation, Report of Crime, Request for Help, Request for Information, Rescue, Response – Community, Response – Formal, Response – Miscellaneous, Response – Personal, Sheltering, Status – Community/Population, Status – Personal, Damage, Status – Infrastructure, Status – Personal Property, Status – Public Property, General Area Information, General Hazard Information, Historical, Prediction, Status – Hazard and lastly, Weather (Vieweg, 2012). With these criteria for classification, there have been studies that capture and process tweets in the wake of natural disasters in general.

One such study adapted the same categories for tweet messages stated earlier. This study's categories for tweet-relevance comprises the following: (1) **Caution and advice**, (2) **Casualties and damage**, (3) **Donations of money, goods or services**, (4) **People missing, found or seen**, and (5) **Information sources**. Along with these adapted categories, the study presented the extraction of information nuggets from microblogging messages during the Joplin tornado of 2011. The study used a crowdsourcing service, *CrowdFlower*, to manually classify tweet messages into a set of classes and to extract short, self-contained structured information that can be used for complex data analysis and integration beyond plain text. The study also employed the use of Naive-Bayesian classifier to do an automated classification of the tweet data (Imran, Elbassuoni, Castillo, Diaz, & Meier, 2013).

Another study used the same criteria stated above to create a mobile application that would determine relevant and irrelevant tweets (Cobo, Parra, & Navon, 2013). However, their adaptation of the criteria did not include the Donations of money, goods or services criteria. In the said study, a classifier was implemented to filter tweets relevant and irrelevant to an earthquake. The data gathered for this study were tweets from the Chilean earthquake of 2010. The initial classification of the tweets according to the situational awareness ontology were done manually by six people.

2.3 Corpus sentiment analysis

There had been studies, which implement the analysis of text data. Most of such studies analyze text-data for any emotions and do automated methods of classification as to what emotion is present in the given corpus.

A study was presented to implement a method, which recognizes emotion using the Knowledge-Based Artificial Neural Networks (K-BANN), and Emotional Keyword Dictionary (EKD) (Arcilla, Amoguis, & Tan, 2013). In this study, corpus, which contained randomly, picked sentences and words were used to train and test the system. Corpus has gone through the evaluation of KBANN and EKD as patterns to determine whether a statement contains any trace of depression.

Another study presented an approach about emotion recognition in Tagalog song lyrics. A hybrid method of an affective Lexicon with Hierarchical Clustering (HCL) and Support Vector Machines (SVM) classifier was used to recognize and classify emotions present in a song lyric (Carreos & Liboon, 2014).

There was a study, which analyzed and classified the behavior of Filipinos on Twitter during a disaster (De Leon, Estuar, Lee, & Ybañez, 2013). The study used Latent Dirichlet Allocation (LDA) and Principal Component Analysis (PCA) to extract and classify the different topics and sentiments present during a disaster. Their findings include what topic and sentiment are more likely retweeted, what language participants use in disasters and primarily, what emotions are present in tweets in times of disasters in the Philippines. However, this study primarily focused on the emotions present in tweet data, and not the relevance of tweets in times of disasters.

2.3.1 Naïve Bayes

Document classification is an important machine-learning concept. In document classification, each instance represents a list of documents, a string of words, or text corpora, along with the instance's class is the document's topic or label. One way of applying machine learning to document classification is to treat the presence or absence of each word as a Boolean attribute. Naive Bayes is a technique best suited for this application due to its very fast and accurate nature (Witten & Frank, 2005).

The Naïve Bayes classifier has been proven to be an effective machine-learning algorithm for sentiment analysis. A comparative study on Multinomial and the multi-variate Bernoulli Naïve Bayes was conducted to prove which of the two variants was effective in effectively class labeling in the context of anti-spam e-mail filtering. The findings of this study stated that Multinomial NB proved to be less biased towards one class or label and achieved slightly higher accuracy than the Bernoulli Naïve Bayes (Schneider, 2003). These two NB variants make assumptions about the distribution of the feature vectors of each document involved. Multinomial NB is often used to classify data based on the frequency of words in a string or document, whereas, Bernoulli NB or *multivariate* Bernoulli NB is used to classify data based on the presence of words or features.

2.3.2 N-Grams

An N-Gram is a sequence of N-words (Kumar, 2017). N-gram models can be described to be a method to predict succeeding words given the previous N-1 words. The Maximum Likelihood Estimation is a probability assigned to N-grams, which is a method that determines values for the parameters of a model. The parameter values are found such that they maximize the likelihood that the process described by the model produced the data that were observed (Brooks-Barlett, 2018).

2.4 Twitter API

Twitter's policy for data sharing is very open, providing a free *Streaming API*, which returns tweets matching a query provided by the Streaming API user. One disadvantage of the Streaming API is that it only returns at most 1% of tweets on Twitter at a given moment. Once the volume of the query surpasses 1% of all the tweets on Twitter, the response is sampled.

2.4.1 Web Applications which uses Twitter's streaming API

There have been several current web applications that display data analytics based on Twitter's streaming API. Tweepchup.com is a website which displays data analytics and infographics on how many mentions or retweets a user gets. It also pinpoints what location did most of a user's mentions come from. This web application also allows users to analyze another user's Twitter data analytics and know how many times certain keywords were mentioned in a tweet within a specified period. However, all these functionalities are made possible with tweets that are publicly available, and not with ones that are set to private.

Another similar web application which uses Twitter's streaming API is Twitonomy.com. It has similar functionalities as Tweepchup.com but gives users an option to download and print the tweet data analytics gathered by the application, if users opt to upgrade to a premium account.

Although these web applications provide forms of analytics to users, these do not include data or tweet news filtering.

2.4.2 Tweepy

Tweepy is an easy to use open-source Python library. Its source code is freely available on GitHub and available in the PIP python package management system (Novalic, 2013). It enables users to communicate with the Twitter API using Python. Tweepy utilizes the OAuth authentication, to provide the following functionalities in the utilization of the Twitter API:

- Tweets can be customized to have a string which identifies the app which was used.
- It doesn't reveal twitter credentials.
- Twitter permissions (e.g., read, write, delete tweets or direct messages) are easier to manage, given that consumer and access tokens and keys are made specifically for a specific use case only (e.g., if the generated tokens and keys are for Reading tweets only, one cannot post tweets).

2.5 Data Visualization

Data Visualization is how humans can understand the results of machine learning and sentiment analysis (Radich, 2017). Effective data visualization should be informative, efficient appealing and interactive (Pittenturf, n.d.). Data visualization gives readers a better understanding and appreciation of how to interpret big data, machine learning, and sentiment analysis.

Since this study involves multi-label tweet data, there is a need to visualize this. A study presented a novel visualization approach for tree data structures. In this paper, large hierarchical data, or trees, were presented using circle packing. Each child node is represented by corresponding circles, within bigger circles that constitute its parent node (Wang, Wang, Dai, & Wang, 2006). The study's circle packing implementation was developed using VC++6.0 and OpenGL.

2.6 Django Web Framework

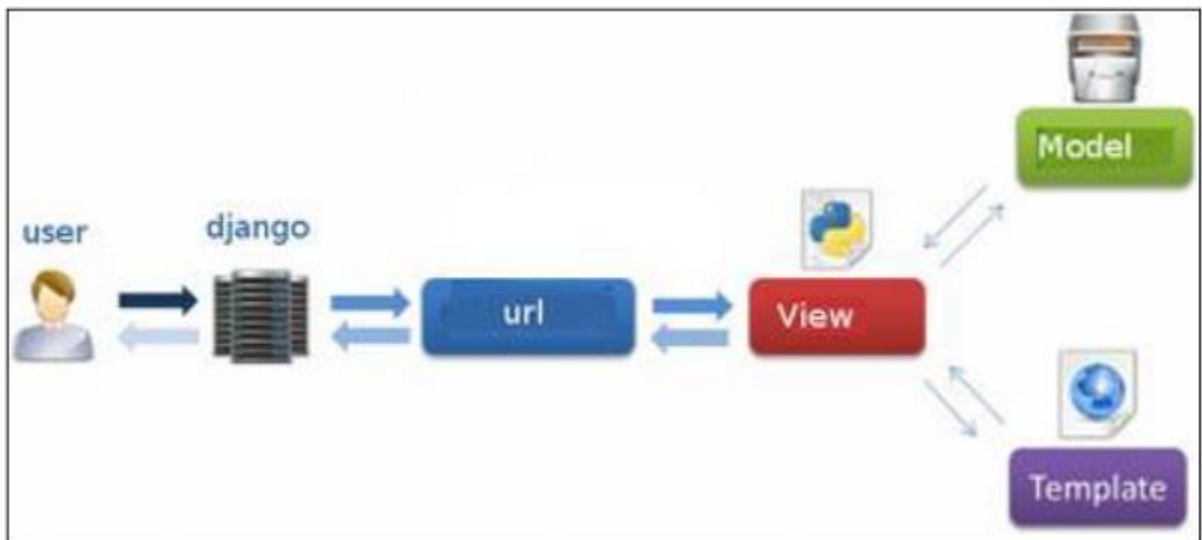


Figure 1: Django's Model-View-Template architecture

Django is a high-level Python Web framework that encourages rapid development and a clean, pragmatic design. This web framework utilizes both the Model-View-Template (MVT) (Django - Overview, 2018). The MVT pattern is different from the traditional Model-View-Controller (MVC) architecture. Models still act as the templates for data, datatypes and the database objects of the system. Django's Views, on the other hand, act as the controller where data are cleaned, filtered and processed. Views also determine the various data be passed on to a template, and which HTML template should be mapped to a URL. Django's template acts as the "View" layer of MVC, where most User

Interface and dynamic Django Template Language (DTL) syntax are encoded for development flexibility. Due to its promise of rapid development, the Django Web framework has been widely used for both organizational and academic implementations.

2.6.1 Python Django for Scientific Web Applications

Python Django had been used for various scientific implementations and visualizations of data from various scientific fields. Two of such implementations which used the Django web framework were in the field of Bio-Chemistry. With Django, two website-tools were created called WONKA and OOMMPPAA (Deanne, Wall, Green, Marsden, & Bradley, 2014). OOMMPPAA is a novel computational tool designed to inform compound design by combining structural and activity data and presents data in an interactive Django application providing users with visual summaries of important interactions of protein-ligand binding sites. WONKA is a tool which assists in uncovering interesting and unusual features within ensembles of protein-ligand structures and enables the facile sharing of observations between scientists (Bradley, et al., 2015).

2.7 Data-Driven Documents (D3.js)

The Data Drive Documents, or D3.js, is a JavaScript library used for manipulating documents based on data. This library implements various data visualization techniques using HTML, SVG, CSS, JavaScript, and JQuery. D3.js' emphasis on web standards enables the full capabilities of modern browsers without the hassle of using proprietary frameworks, by combining powerful visualization components and a data-driven approach to Document Object Model (DOM) manipulation (Data-Driven Documents, 2018).

2.7.1 Circle Packing

Circle packing is an arrangement of circles within a set boundary or shape (Weisstein, n.d). The circles are packed together in a manner that these do not overlap one another, but instead, are rearranged symmetrically, and are resized based on the population of a class that a circle represents.

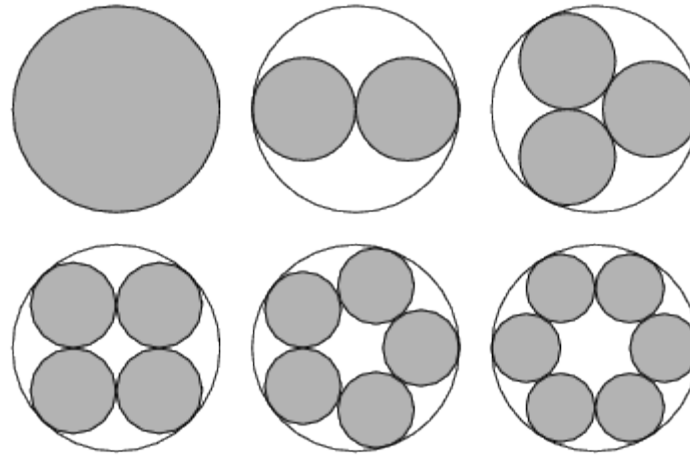


Figure 2: Circle Packing

A study utilized the library called Data-Driven Documents (D3js) to visualize twitter hashtags present in the #BostonMarathon event. D3js' circle packing implementation was used to visualize what were the most and least prominent words used during the #BostonMarathon event (Salarpour, Bamneshin, & Proios, 2014).

CHAPTER 3

THEORETICAL FRAMEWORK

A study on the practical extraction of information presented this framework which uses the situational awareness ontology in the manual pre-classification of storm-related tweet data and the automated extraction of class-related information (Imran, Elbassuoni, Castillo, Diaz, & Meier, 2013). It had four steps namely: Gather storm-related tweets using Twitter’s streaming API, Manual classification of tweets, Automatic classification of tweets, and the Evaluation of their system by actual users.

3.1 Gather the storm-related tweets

The initial step was to gather storm-related tweets using Twitter’s streaming API. Two datasets were gathered, tweets during the tornado that struck Joplin, Missouri on May 22, 2011, and tweets during the Hurricane Sandy, which hit Northeastern US on October 29, 2012. The two datasets were gathered using Twitter’s streaming API through the hashtags #joplin and #sandy respectively.

3.2 Manual classification of tweets

The second step was the classification of the tweets, as most tweet messages generated during a disaster are extremely varied and free-flowing. The dataset was both classified manually and automatically. The manual classification process used was crowdsourcing (using *CrowdFlower*), and each assessor was shown a tweet, along with instruction and an empty text input field and was asked to copy-paste a word or short phrase from the tweet conveying the specified information. Tweets were manually classified thrice based on the criteria and ontologies derived from previous studies.

(1) Initial classification task was to filter informative messages based on three main classes: Personal – messages, which were only of interest to its author and his/her immediate circle of family/friends; Informative – messages of interest to other people beyond the author’s immediate circle; and Other – messages that are not related to the disaster at all.

(2) Only Informative tweet messages were considered, and were classified again by type based on the following classes: Caution and Advice – messages that convey or reports information about some warning or piece of advice about a possible hazard of an incident; Casualties and Damage – messages that report about casualties or infrastructure damage done by an incident; Donations – messages about goods or services offered or needed by the victims of an incident;

People – messages about missing or found persons that were affected by an incident, or reports the reaction or visit of a celebrity; Information Sources – messages that point to information sources, photos, videos or mentions a website, TV or radio station providing extensive coverage; and Other – which conveys messages about other types of informative messages.

(3) For each of the said types, a separate crowdsourcing task was created where workers were asked to classify a message by sub-type and were asked to extract a sub-string from the message containing a given piece of information.

3.3 Automatic classification of tweets

The automated classification process involved training a set of multi-label classifiers, using Naïve Bayesian classifiers implemented in Weka, where some binary, scalar and text features were employed. The binary features determined whether a tweet contained the following features or not: the @ symbol, a URL, a *hashtag*, an *emoticon*, and a number. The scalar feature implied only the tweet length. While, text features excluded non-words from the binary features, and extracted the following: unigrams, bigrams, Part of Speech (POS) tags, POS tag-bigrams, and VerbNet classes. The classifier repeated the three tasks stated above. This was made possible by training the classifier using the data gathered from the manual classification method.

3.4 Evaluation

A set of training tweets was used to task *CrowdFlower* workers to extract the desired information manually and to measure the quality of the information extracted. The same information was also automatically extracted from the same tweets using extracted data. The following metrics were used to evaluate the performance of the classifiers: Area Under the Curve (AUC), Precision, Recall, and F-Measure. The *Hit-Ratio* was formulated to determine the accuracy of the extracted information of the automatic classifier. The *Hit-Ratio* is defined as:

$$hit - ratio = \frac{\sum_{i=1}^{|tweets|} hit_i}{|tweets|}$$

Where $|tweets|$ is the total number of tweets for which the information from classified tweets was manually extracted and,

$$hit_i = \begin{cases} 1 & \text{if } editdistance(manual_i, auto_i) \leq 2 \\ 0 & \text{otherwise} \end{cases}$$

$manual_i$ is the manually extracted information from tweet t_i and $auto_i$ the automatically extracted information from tweet t_i .

CHAPTER 4

RESEARCH DESIGN AND METHODOLOGY

4.1 Conceptual Framework

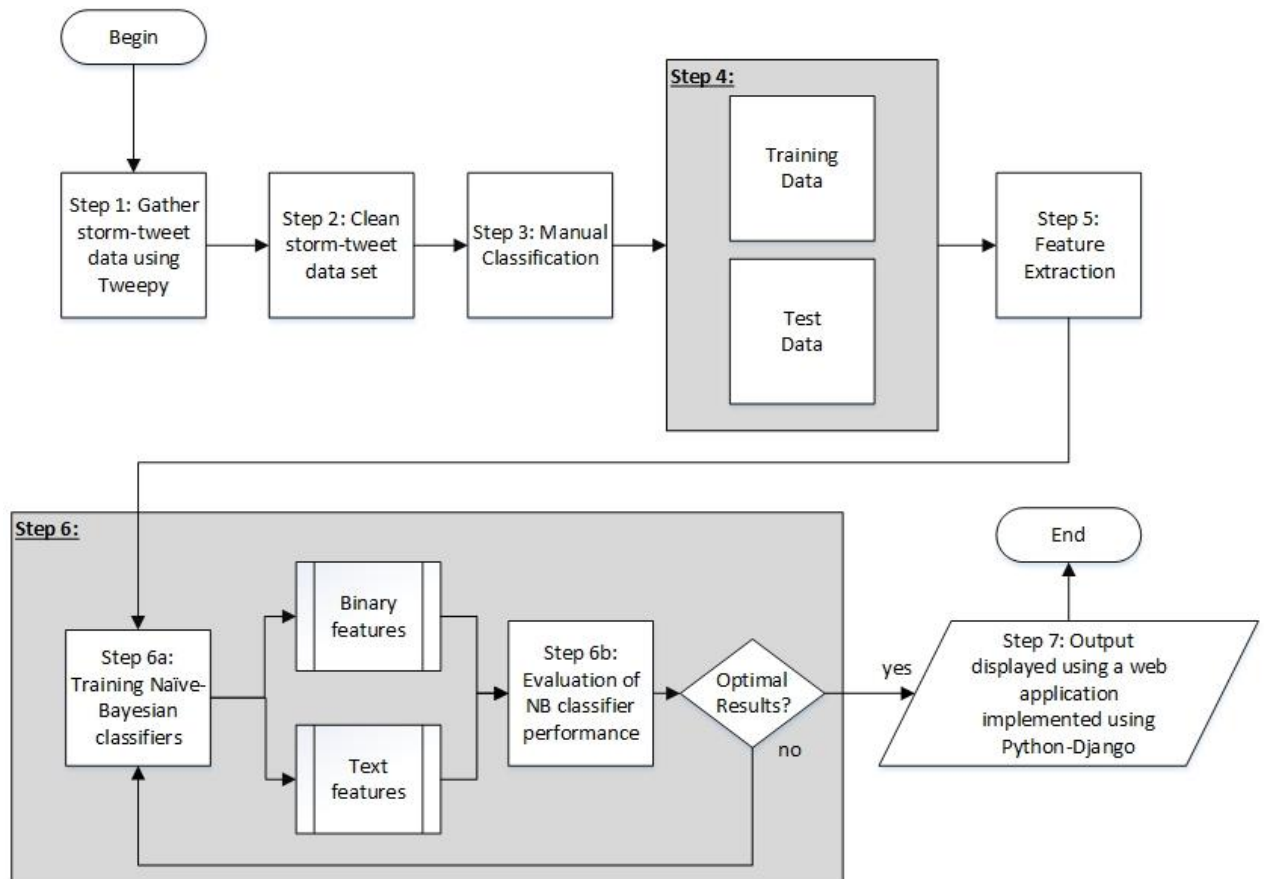


Figure 3: The Conceptual Framework

4.2 Methodology

4.2.1 Gather Tweet Dataset

A python script was written to gather storm-tweet data. The script utilized two python libraries: Tweepy and CSV. Tweepy is a pythonic library which interfaces with twitter's restful API. With Tweepy, various twitter actions can be made such as, looking up public tweets, writing tweets and deleting personal tweets. This is made possible by instantiating an OAuth object, which takes in a consumer API key, and a consumer API secret key which are generated from

<https://apps.twitter.com>. The OAuth object was then assigned to a local variable named API and was used as a personal “key” in doing back-end interactions with the Twitter restful API. With the API variable, the script could do a twitter search query, represented by the variable q. These variables were passed into the parameter of the Cursor function. The Cursor function is a Tweepy object used to make pagination easier.

The python CSV library allows create, read, update and delete (CRUD) interactions with comma-separated value files. With the CSV library, each tweet feature and object instance was written into a CSV file.

```
# script which gathers tweets containing 'nonaPH'
# Ogs Ablazo
import tweepy, csv

namefile = "nonaPHtweets.csv"

consumer_key = #consumerkey
consumer_secret = #consumersecret

auth = tweepy.OAuthHandler(consumer_key = consumer_key,
                           consumer_secret = consumer_secret)
api = tweepy.API(auth)
csvFile = open(namefile, 'a')
csvWriter = csv.writer(csvFile)
csvWriter.writerow(['username', 'date created', 'retweets',
                    'favorites', 'tweet text', 'location', 'id'])

for tweet in tweepy.Cursor(api.search q='nonaPH').items():
    csvWriter.writerow([tweet.user.screen_name,
                        tweet.created_at,
                        tweet.retweet_count,
                        tweet.favorite_count,
                        tweet.text.encode('utf8'),
                        tweet.user.location.encode('utf8'),
                        tweet.user.id])

csvFile.close()
```

Figure 4: Script for collecting nonaPH tweets using Tweepy

On December 14, 2015, the typhoon Nona (international name: Melor) was observed entering the Philippine Area of Responsibility and had landfall on Batag Island of Laoang, Northern Samar Philippines. The script was initially executed from December 12, 2015, until the storm dissipated on December 31, 2015. The script gathered tweets bearing the hashtag #NonaPH and gathered a total of 14685 tweets. The typhoon had landfall on December 14 and

dissipated on December 17. Therefore, the dataset used only focused on the tweets between those specific dates, which totaled 12385 tweets.

The following tweet features were part of the tweets rows gathered:

1. **username** – this tweet feature indicates the Twitter user’s username.
2. **Date created** – this tweet feature indicates the date and time when the tweet was posted on Twitter.
3. **Retweet count**– this tweet feature indicates the total number of retweets a tweet has amounted to now that the tweet was gathered by the script.
4. **Favorite count** - this tweet feature indicates the total number of favorites a tweet has amounted to now that the script gathered the tweet.
5. **Tweet text** – this tweet feature contains the tweet corpus.
6. **Location** – this tweet feature contains the location of the tweeting user. If the value is NONE, then the twitter user hasn’t registered a location or place of origin in his or her profile
7. **id** – this tweet feature contains a unique numeric ID which uniquely identifies the tweet.

4.2.2 Cleaning of Tweet Dataset

Before any classification was done, the dataset was cleaned of any noise. Noise meant any instances of the following: symbols (non-alphanumeric characters), URLs (which also included links to photos), mentions (*@username*), and hashtags were removed. Retweets and duplicate tweets were also removed because these tweets would affect the accuracy of the automatic classifiers later. Blank columns and cells were also replaced with ‘none’ values – to avoid further errors as the data would be fed in the automatic classifiers.

A script was made to remove non-alphanumeric symbols from the tweet dataset. Only the python CSV library was imported in the script to handle the .csv file containing the tweets. Each tweet instance was stripped of the following non-alphanumeric characters: `_"$.%@#!^&*() - += , ; : [ ] { } | .`

```

#script to remove the following symbols
#in tweet text:
#_"/.$%#@#!^&*()-+=,;[{}]|

import csv

with open("rangedData.csv", "rb") as infile, open("repaired.csv", "wb") as outfile:
    reader = csv.reader(infile)
    writer = csv.writer(outfile)
    conversion = set('_"/.$%#@#!^&*()-+=,;[{}]|')
    for row in reader:
        newrow = [''.join('' if c in conversion else c for c in entry) for entry in row]
        writer.writerow(newrow)

```

Figure 5: Python script used to remove non-alphanumeric symbols from the tweet dataset

This script (fig 6) programmatically removes all instances of hyperlinks, user mentions and hashtags in tweets. The RE and CSV python libraries were used in this script. The RE python library contains all Regular Expression operations and methods.

```

#script to remove the following in tweets:
#http:// and https:// (urls), mentions and hashtags

import re
import sys
import csv

def data_clean(fn):
    fin = open(fn, 'rb')
    fout = open("cleaned.csv", 'wb')
    results = []
    patterns = [ r"http:\/\/\w+\S+", r"@w+", r"#w+" ]
    for text in csv.reader(fin):
        newrow = [''.join('' if c in conversion else c for c in entry) for entry in row]
        csv.writer(fout).writerow(newrow)

text_file = sys.argv[1]
data_clean(text_file)

```

Figure 6: Python script used to remove URLs, mentions, and hashtags from the tweet dataset

A script (fig 7) was made to replace all blank cells with the 'NONE' string for each tweet instance. This task was done because there were tweet features that don't contain any value. Once these blank cells are updated with a 'NONE' string, these are reinserted into the tweet instance it belonged to.

```

# Ogs Ablazo
# it is assumed that the data set has been cleaned of hashtags,
# symbols, and URLs before executing this script.
# this does the following:
# 1 - reads a 'clean' csv.
# 2 - reads each line from the csv.
# 3 - replaces all blank cells in the current line iteration, with the string "NONE"
# 4 - writes each updated line into a new row in a new csv file
import sys
import csv
import fileinput
import string

in_file = sys.argv[1]
row_reader = csv.reader(open(in_file,"rb"))
row_writer = csv.writer(open("noRTnoBlankData.csv", "wb"))

first_row = row_reader.next()
row_writer.writerow(first_row)

for row in row_reader:
    new_row = [val if val else "NONE" for val in row] + (["NONE"] * (len(first_row)-len(row)))
    row_writer.writerow(new_row)

```

Figure 7: Python script used to replace blank cells in the tweet dataset with 'none'

A script (fig 8) to detect and remove Retweets was also made. All retweeted tweet text instances in Twitter are prefixed with the 'RT' label or the 'Retweeted' string, but these prefixes are only visible in downloaded data sets, not in the Twitter feed. These were removed from the dataset, to avoid any inconsistencies on the classification accuracy.

```

#Ogs Ablazo
# this script takes in a csv file (i.e. ranged.csv)
# and removes tweet texts that start wt RT or Retweeted
# any RT or Retweeted preceded by another group of words is
# a 'Quote' (quoted retweet). not part of the removal process

import sys
import fileinput
import csv

file_name = sys.argv[1]

rta = retweeteda = nort = 0

with open(file_name, 'rU') as in_file, open('rtfiltered.csv','wb') as out_file:
    writer = csv.writer(out_file)
    for row in csv.reader(in_file, delimiter = ","):

        if row[5].startswith('RT'):
            print row[5]
            rta = rta + 1
        elif row[5].startswith('Retweeted'):
            print row[5]
            retweeteda = retweeteda + 1
        else:
            nort = nort + 1
            writer.writerow(row)

```

Figure 8: a Python script to remove duplicates and retweets from the tweet dataset

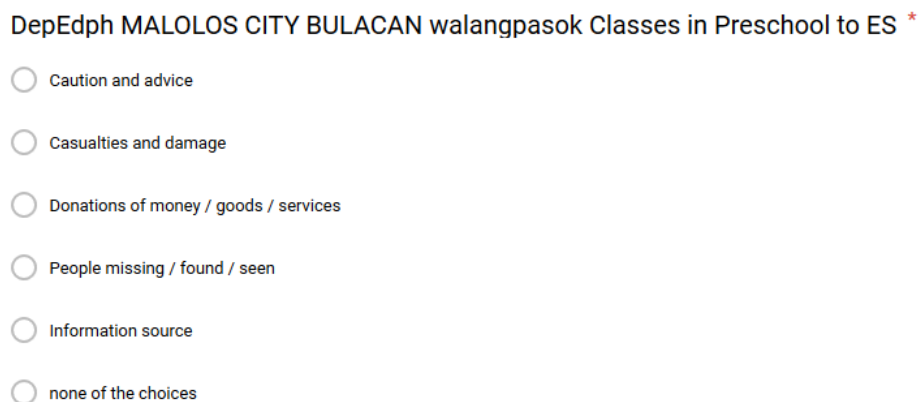
After cleaning the nonaPH tweet dataset, a total of 12054 tweets were prepared for manual classification.

4.2.3 Manual Classification of Tweets

Using Google forms, crowdsourced respondents were tasked to do the manual classification of each tweet corpora to attain training data that were used to train Automatic classifiers. The annotators were undergraduate students from the Ateneo de Davao University, aged 18 to 20 years old. Two hundred fifty crowdsourced annotators labeled the cleaned tweet dataset three times. These annotators were tasked to label survey sets that contained 144 to 145 tweet texts. All annotators were briefed of the following tweet categories before further classification was done:

- **Caution and Advice:** if a tweet conveys/reports information about a warning or a piece of advice about a possible hazard of an incident.
- **Casualties and damage:** if a tweet reports the information about casualties or damage done by an incident.
- **Donations of money, goods or services:** if a message speaks about money raised, donation offers, goods/services offered or asked by the victims of an incident.
- **People missing, found or seen:** if a message reports about the missing or found person affected by an incident or seen a celebrity visit on ground zero.
- **Information source:** if a message conveys/contains some information sources like a photo, footage video, or mentions other sources like TV, radio related to an incident.

However, if the tweet didn't belong to the said tweet categories, the annotators would then opt to choose the “**none of the choices**” category. The none of the choices option – signifies that the tweet is not relevant, or not informative.



DepEdph MALOLOS CITY BULACAN walangpasok Classes in Preschool to ES *

☐ Caution and advice

☐ Casualties and damage

☐ Donations of money / goods / services

☐ People missing / found / seen

☐ Information source

☐ none of the choices

Figure 9: View of sample tweet survey with six categories in Google forms

All 12054 tweets were classified in three batches. Two hundred fifty survey sets consisting of 144-145 tweet texts were given to the 250 annotators. The tweet

data set was labeled in three batches because at least two people had to agree on a tweet classification label.

A script (fig 10) was made to determine which tweets had two to three label agreements between three batches. The script contained the `define_class()` method which compared each tweet label. If there were two to three similar labels – then that would be the defined classification for the said tweet row. However, if there were no similar labels, a ‘conflicting’ string would be returned. By the end of the script, all tweets that had a common class would have their labels saved in a CSV column named ‘defined_class’.

```
import csv, pandas as pd, numpy as np

df = pd.read_csv("consdata.csv")

def define_class(x,y,z):
    if (x == y) or (y == z) or (z == x):
        if x == y:
            def_class = x
        if y == z:
            def_class = y
        if z == x:
            def_class = z
        return def_class.lower().strip(' /')
    else:
        return 'conflicting'

df['defined_class'] = try_class = ""
c = d = 0

for i, row in df.iterrows():
    try_class = define_class(row['tweet_class1st'].lower().strip(' /'),
                             row['tweet_class2nd'].lower().strip(' /'),
                             row['tweet_class3rd'].lower().strip(' /'))

    if try_class == 'conflicting':
        d = d + 1
    else:
        c = c + 1
    df.at[i, 'defined_class'] = try_class
print "defined classes %s" %c
print "conflicting classes: %s" %d
df = df[df.defined_class!='conflicting'] #remove conflicting classes
df.to_csv('common_classes.csv')
print len(df)
```

Figure 10: Script to remove rows that didn't have label agreements

Out of the 12054 tweets, a total of 11340 tweets contained 2 to 3 labels that matched. 1089 tweets were classified under *Caution and advice*, 434 tweets were classified under *Casualties and damage*, 185 tweets under *Donations of money goods services*, 62 tweets under *People missing found or seen*, 3929 tweets under *Information source* and 5641 tweets under the *none of the choices*

category. 714 tweets didn't contain a matched or similar label for the three batches, and therefore, these were discarded.

username	date_created	retweets	favorites	tweet_text	location	tweet_id	tweet_class1st	tweet_class2nd	tweet_class3rd	defined_class
MichelleMorente	20151217 2355	19	130	Pinamalayan M	NONE	6.78E+17	none of the choices	none of the choices	none of the choices	none of the choices
ElmaGreenwood	20151217 2351	0	0	From Allen Noi	NONE	6.78E+17	Donations of money g	none of the choices	none of the choices	none of the choices
mercadoxcarl	20151217 2350	0	3	how to stop yo	NONE	6.78E+17	none of the choices	none of the choices	none of the choices	none of the choices
PHINews1	20151217 2347	0	0	P57M worth of	NONE	6.78E+17	Donations of money g	Casualties and damage	Casualties and damage	casualties and damage
PHINews1	20151217 2347	0	0	Downed comm	NONE	6.78E+17	Casualties and damage	Information source	Casualties and damage	casualties and damage
aivancalonia	20151217 2343	0	0	Polluters indus	NONE	6.78E+17	Casualties and damage	none of the choices	none of the choices	none of the choices
PHINews1	20151217 2339	0	1	Santa Claus car	NONE	6.78E+17	none of the choices	none of the choices	none of the choices	none of the choices

Figure 11: Snippet of the dataset with the corresponding defined_class values

4.2.3.1 Multi-rater Reliability using Fleiss' Kappa

Fleiss' kappa is a statistical measure used to evaluate the reliability of agreement between 3 to many raters when assigning a categorical rating to several items. Fleiss' Kappa is defined as:

$$\kappa = \frac{\bar{P} - \bar{P}_e}{(1 - \bar{P}_e)}$$

$\bar{P}$ is the mean of all of all the raters' agreed ratings, and $\bar{P}_e$ is the mean of all the rater's ratings agreeing solely due to chance. Subtracted, $\bar{P} - \bar{P}_e$ gives the degree of agreement of ratings achieved above chance, and $1 - \bar{P}_e$ gives the degree of agreement that is attainable above chance. If $\kappa = 1$, then the raters are in complete agreement, else there is no agreement between raters if $\kappa \leq 0$.

For this study's manually classified data, the *raters* are the three batches of annotators that labeled the data three times. The achieved κ value was computed at 0.53609. This implies that there was a moderate agreement between raters. This interpretation was based on a study, which discussed the interpretations between κ value ranges (Landis & Koch, 1977).

κ	Interpretation
< 0	Poor Agreement
$0.0 - 0.20$	Slight Agreement
$0.21 - 0.40$	Fair Agreement
$0.41 - 0.60$	Moderate Agreement
$0.61 - 0.80$	Substantial Agreement
$0.81 - 1.0$	Almost Perfect Agreement

Table 1: Landis and Koch's Fleiss Kappa interpretation

4.2.4 Data Split

Before feeding the dataset into the classifiers, these were split into training and testing data. The training data was used to train and fit the classifiers based on tweet features (n-grams). Whereas the testing data was used to test the accuracy of the Naïve Bayes classifiers on how well it predicted or classified tweets. Scikit learn's `train_test_split` module was used to split the dataset into random training and testing subsets. The data was split into two sets of experiments: 1) For all six classes, and, 2) For the two main classes.

4.2.4.1 Data Splitting for 6 Classes

The classes used for the data are the six sub-classes of tweet labels from the *defined_class* column: caution and advice, casualties and damage, information source, donations of money goods services, people missing found seen, and, none of the choices. Due to the aforementioned classes' imbalanced counts, these class was undersampled so that a balanced data set would be attained. Undersampling is the process of reducing the total number of instances of a class to match the class with the least number of instances. This process was done to attain a balanced data set. Pandas data frame's `.sample()` method was used to undersample each of the classes. The `.sample()` method accepts one integer parameter, which is the total count by which a category is undersampled. It would then return randomly picked sample rows from a class' data frame amounting to the integer parameter set in `.sample()`.

```
notc.sample(min_sample), info.sample(min_sample),  
caa.sample(min_sample), cad.sample(min_sample),  
domgs.sample(min_sample)
```

Figure 12: Script to randomly under sample each class

The least numbered category was people missing found seen which totaled at 62 instances. Therefore, each class was set to 62 instances each.

After undersampling each of the tweet classes, these were subject to data splitting. Sklearn.model_selection's `train_test_split` was used to split the data into the training and test datasets. The `train_test_split` module accepted five arguments. `x` – constitutes the cleaned tweet text dataset — `y` – which constitutes the Boolean values of the *isInformative* feature associated with each tweet text data. `train_size` – indicates the training size percentage, for this study, 75% (279 tweets) of the overall tweet dataset was used as training data, while 25% (93 tweets) for testing data and `random_state` indicates the random seed level.

```
x, y = df_under.tweet_text, df_under.defined_class
x_train, x_test, y_train, y_test = train_test_split(x, y, train_size=0.75, random_state=10)
```

Figure 13: Script to split and assign the 6-class dataset into its train and test datasets

The result of the `train_test_split` module would be the respective training and testing data for both tweet text and the corresponding ground truth label.

4.2.4.2 Data Splitting for 2 Classes

The two main classes, *Informative* and *Not Informative*, were also split to determine whether these would perform better in the Bayesian classifiers. Informative labels have a total of 5699 instances, and Not Informative labels have a total of 5641 instances, these instances are balanced, and need not be adjusted or undersampled, unlike the six individual sub-classes.

```
df1 = pd.read_csv("hasNoClassmention_dataset.csv")

x, y = df1.tweet_text, label_binarize(df1.isInformative,
                                      classes=["notinformative", "informative"])
x_train, x_test, y_train, y_test = train_test_split(x, y,
                                                    stratify=y,
                                                    train_size=0.75,
                                                    random_state=10)
```

Figure 14: Script to split and assign the 2-class dataset into its train and test datasets

The `train_test_split` module accepted five arguments. Along with the `x`, `y`, `train_size`, and `random_state` arguments, the `stratify=y` keyword-argument was added. The keyword argument, `y`, does a split so that the proportion of values in the sample produced will be the same as the proportion of values provided in `y`.

The `train_size` – for this data set amounted to 75% (8,505 tweets) of the overall tweet dataset was used as training data, while 25% (2,835 tweets) for testing data. The result of the `train_test_split` module would be the respective training and testing data for both tweet text and the corresponding ground truth label (informative or not).

4.2.5 Feature Extraction

Feature Extraction is the process of transforming both the training and testing text data into numerical features which are usable by machine learning algorithms such as Naïve Bayes. These text data were vectorized and transformed respectively by the `CountVectorizer` and `TfidfTransformer`

modules of `sklearn.feature_extraction.text`. The output of the `CountVectorizer` module is a matrix of token counts which is fed to the `TfidfTransformer`, which produces Numpy array which contained numerical features derived from the non-numerical and arbitrary data of the tweet data set.

Sklearn's `CountVectorizer()` converts a collection of text documents or strings to a matrix of token counts. For this implementation, the `CountVectorizer` accepts and uses the following parameters: `gram_range` and `stop_words`. The `gram_range` parameter accepts a tuple python sequence, which contains two values: the lower and upper boundary of the range of n-values for different n-grams to be extracted (i.e., a minimum of 2 and maximum of 7 n-grams).

The data is fed to `CountVectorizer` by using the method `.fit_transform()`. This method accepts the training text dataset. The `.fit_transform()` method learns the vocabulary dictionary based on the dataset, the `gram_range` values, which stop words to omit and returns a term-document matrix. The term-document matrix is a *tally* of all instances of terms from the data set for each tweet text instance, where zero (0) indicates an absence of a said term otherwise, a positive non-zero value would indicate the presence of a said term. In this implementation, a *term* is an n-gram between 1 to 6-grams or 1 to 6 adjacent word combinations.

```
vect = CountVectorizer(ngram_range=(1,6), stop_words=tl_enSW)
vect_ft = vect.fit_transform(x_train.head(2))
print(vect.get_feature_names())
```

```
[ '16', '16 ty', '16 ty nonaph', '16 ty nonaph http', '16 ty nonaph http bitlylo2i6qn', '16 ty nonaph http bitlylo2i6qn cnn', 'bitlylo2i6qn', 'bitlylo2i6qn cnn', 'bitlylo2i6qn cnn philippines', 'bitlylo2i6qn cnn philippines cnnphilippines', 'bitlylo2i6qn cnn philippines cnnphilippines dec', 'class', 'class suspensions', 'class suspensions december', 'class suspensions december 16', 'class suspensions december 16 ty', 'class suspensions december 16 ty nonaph', 'cnn', 'cnn philippines', 'cnn philippines cnnphilippines', 'cnn philippines cnnphilippines dec', 'cnnphilippines', 'cnnphilippines dec', 'dec', 'december', 'december 16', 'december 16 ty', 'december 16 ty nonaph', 'december 16 ty nonaph http', 'december 16 ty nonaph http bitlylo2i6qn', 'end', 'end year', 'end year party', 'end year party huy', 'ezhan', 'ezhan mo', 'ezhan mo ha', 'ezhan mo ha end', 'ezhan mo ha end year', 'ezhan mo ha end year party', 'ha', 'ha end', 'ha end year', 'ha end year party', 'ha end year party huy', 'http', 'http bitlylo2i6qn', 'http bitlylo2i6qn cnn', 'http bitlylo2i6qn cnn philippines', 'http bitlylo2i6qn cnn philippines cnnphilippines', 'http bitlylo2i6qn cnn philippines cnnphilippines dec', 'huy', 'mo', 'mo ha', 'mo ha end', 'mo ha end year', 'mo ha end year party', 'mo ha end year party huy', 'nonaph', 'nonaph ezhan', 'nonaph ezhan mo', 'nonaph ezhan mo ha', 'nonaph ezhan mo ha end', 'nonaph ezhan mo ha end year', 'nonaph http', 'nonaph http bitlylo2i6qn', 'nonaph http bitlylo2i6qn cnn', 'nonaph http bitlylo2i6qn cnn philippines', 'nonaph http bitlylo2i6qn cnn philippines cnnphilippines', 'party', 'party huy', 'philippines', 'philippines cnnphilippines', 'philippines cnnphilippines dec', 'suspensions', 'suspensions december', 'suspensions december 16', 'suspensions december 16 ty', 'suspensions december 16 ty nonaph', 'suspensions december 16 ty nonaph http', 'ty', 'ty nonaph', 'ty nonaph http', 'ty nonaph http bitlylo2i6qn', 'ty nonaph http bitlylo2i6qn cnn', 'ty nonaph http bitlylo2i6qn cnn philippines', 'updated', 'updated walangpasok', 'updated walangpasok class', 'updated walangpasok class suspensions', 'updated walangpasok class suspensions december', 'updated walangpasok class suspensions december 16', 'walangpasok', 'walangpasok class', 'walangpasok class suspensions', 'walangpasok class suspensions december', 'walangpasok class suspensions december 16', 'walangpasok class suspensions december 16 ty', 'year', 'year party', 'year party huy']
```

Figure 15: Features generated by `fit_transform` for the first two rows of the training data set

```
print(vect_ft.toarray())
```

```
[ [1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 0 0 0 0  
0 0 0 0 0 0 0 0 0 1 1 1 1 1 0 0 0 0 0 0 1 0 0 0 0 0 1 1 1 1 0 0 1  
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 0]  
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1  
1 1 1 1 1 1 1 1 1 0 0 0 0 0 0 1 1 1 1 1 1 1 0 0 0 0 0 1 1 0  
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1]]
```

Figure 16: The term-document matrix of the first two rows of the training dataset.

After getting the term-document matrix for the entire training dataset, the matrix is then fed to Sklearn's `TfidfTransformer()` method. `TfidfTransformer()` transforms the count matrix result of `CountVectorizer` into a normalized term-frequency (TF). The term frequency-inverse document frequency (TFIDF) is a method used to determine how important a word or an n-gram is to a given text corpus.

```
tran = TfidfTransformer()
tran_ft = tran.fit_transform(vect_ft)
print(tran_ft.toarray())
```

```
[[0.12081891 0.12081891 0.12081891 0.12081891 0.12081891 0.12081891
 0.12081891 0.12081891 0.12081891 0.12081891 0.12081891 0.12081891
 0.12081891 0.12081891 0.12081891 0.12081891 0.12081891 0.12081891
 0.12081891 0.12081891 0.12081891 0.12081891 0.12081891 0.12081891
 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0.12081891 0.12081891 0.12081891
 0.12081891 0.12081891 0.12081891 0. 0. 0.
 0. 0. 0. 0. 0.08596365 0.
 0. 0. 0. 0. 0.12081891 0.12081891
 0.12081891 0.12081891 0.12081891 0. 0. 0.12081891
 0.12081891 0.12081891 0.12081891 0.12081891 0.12081891 0.12081891
 0.12081891 0.12081891 0.12081891 0.12081891 0.12081891 0.12081891
 0.12081891 0.12081891 0.12081891 0.12081891 0.12081891 0.12081891
 0.12081891 0.12081891 0. 0. 0. ]
[0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0.
 0.17539476 0.17539476 0.17539476 0.17539476 0.17539476 0.17539476
 0.17539476 0.17539476 0.17539476 0.17539476 0.17539476 0.17539476
 0.17539476 0.17539476 0.17539476 0. 0. 0.
 0. 0. 0. 0.17539476 0.17539476 0.17539476
 0.17539476 0.17539476 0.17539476 0.17539476 0.12479481 0.17539476
 0.17539476 0.17539476 0.17539476 0.17539476 0. 0.
 0. 0. 0. 0.17539476 0.17539476 0.
 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0.
 0. 0. 0.17539476 0.17539476 0.17539476]]
```

Figure 17: The TF-IDF representation of the first two rows of the training dataset.

4.2.6 Creating the automatic classifiers

The BernoulliNB and MultinomialNB methods were utilized from Sklearn, which is a library in Python (SciKit Learn) primarily used for data science.

Sklearn's Pipeline function was used to simplify the feature extraction and classifier code further and contain the entirety of the classifier models. The Pipeline function sequentially applies a python list of transformers and vectorizers and a final estimator or classifier. Transformers and vectorizers mold the data to be classified (i.e., transform text data into numeric data) so that these would be compatible with an estimator to be used. The Estimators are rules for calculating an estimate of a given quantity based on observed data, features, or labels.

Within the Pipeline's parameters, is a pythonic list containing the following tuple pairs: *vect* and CountVectorizer(), *tfidf* and TfidfTransformer(), and, *clf* and MultinomialNB() / BernoulliNB(). Each tuple pair observes a key *implies* value relationship, such that each value would then be referenced later on, by calling its respective key.

For this study, the ngram_ranges of (1, 3) to (4, 6) were the features tested on by the Pipeline. This meant that a minimum of 1 to a maximum of 3 n-grams ranges to a minimum of 4 to a maximum of 6 n-gram ranges were the considered features for each NB estimator. This was done to identify how well each classifier faired with little n-gram range to bigger n-gram range combinations. The stop_words parameter accepts a list of commonly used words which are to be filtered from the text corpora. The list of stop words is a mix of both Tagalog and English stop words. These can be found in the appendices.

```
i, j = 1, 4
for j in range(4,7):
    for i in range(1,4):
        text_clf = Pipeline([('vect', CountVectorizer(ngram_range=(i,j),
                                                         stop_words=tl_enSW)),
                             ('tfidf', TfidfTransformer()),
                             ('clf', MultinomialNB()),])
```

Figure 18: Script to initialize the Pipeline, with varying n-gram features

```
text_clf.fit(x_train,y_train)
y_pred = text_clf.predict(x_test)
```

Figure 19: Script to train the datasets and generate a prediction

The respective Pipelines were assigned to a variable called `text_clf` which was used alongside the `.fit()` and `.predict()` methods. The `fit()` method accepts the parameters: `x_train` and `y_train`, which contains the training tweet text data and labels respectively. Sklearn's fit method learns from the training dataset fed into its parameters. The `.fit()` method looks for coefficients for the equation specified in the algorithm/estimator indicated in the Pipeline which is either – MultinomialNB or BernoulliNB. Based on what was learned and trained from the `.fit()` method, `.predict()` attempts to classify testing data_sets, which are contained in the `x_test` variable. The corresponding classifications are then saved in `y_pred`, which are used for further evaluation.

4.2.7 Evaluation

After the automatic classifiers classified tweets, these were evaluated according to the `accuracy_score` and the following classification metrics from the sklearn library: Precision, Recall and F-measure (or F1-score). Respective confusion matrices and Receiver Operating Characteristic-Area Under the Curve (ROC-AUC) diagrams were also generated to further evaluate the efficiency of each Naïve Bayes implementation on the datasets.

The `accuracy_score` method computes subset accuracy. If the entire dataset of predicted labels for a sample exactly matches with the ground truth of labels, then the accuracy would be 1.0 (or 100%). The accuracy score is computed using the following formula:

$$\text{accuracy}(y, \hat{y}) = \frac{1}{n_{\text{samples}}} \sum_{i=0}^{n_{\text{samples}}-1} 1(\hat{y}_i = y_i)$$

The `accuracy_score` method accepts two parameters which are `y_test` and `y_pred` and returns a floating point.

Sets of accuracy scores and classification results were derived from the n-gram ranges of (1,3) to (4,6).

```
text_clf.fit(x_train,y_train)
y_pred = text_clf.predict(x_test)
print("accuracy score %s"%accuracy_score(y_test,y_pred))
print(classification_report(y_test, y_pred))
```

Figure 20: Script to determine accuracy score and classification report.

The `classification_report` method accepts two parameters. Like `accuracy_score`, it also accepts `y_test` and `y_pred`, from which it computes for the precision, recall, and f1-score of the classification model. The `classification_report` method then returns a table containing the computed precision, recall, f1-score and the support involved behind each class in the dataset. Precision is the ratio of the correctly predicted labels and the total predicted labels. In other studies, this is also known as the *hit ratio*. This is calculated by dividing the total number of True Positives by the sum of True and False Positives.

$$Precision = \frac{True\ Positives}{True\ Positives + False\ Positives}$$

The recall is the ratio of correctly labeled text data and the actual number of correct labels, in other studies this is also known as the *detection rate*. This is calculated by dividing the total number of True Positives by the sum of True Positives and False Negatives.

$$Recall = \frac{True\ Positives}{True\ Positives + False\ Negatives}$$

The F1-score or the *F-measure* is the harmonic mean of both precision and recall scores, which is derived by using the following formula:

$$F1score = 2 \times \frac{Precision * Recall}{Precision + Recall}$$

Support merely identifies how many instances were sampled and labeled for testing.

CHAPTER 5

RESULTS AND DISCUSSION

5.1 Comparison of the Evaluation Results

For the 6-Class dataset, MNB achieved its highest accuracy score of 0.63441 at n-gram ranges (1,4), and, (1,5), over the BernoulliNB's highest accuracy score of 0.30108 at n-gram range (1,4). The MNB having its highest Precision score average of 0.72 at n-gram ranges (1,4) and, (1,5), over the BNB's highest Precision score average of 0.825 at n-gram ranges (1,5), (1,6), and, (2,6). The highest MNB Recall average score of 0.63 at n-gram ranges (1,4) and, (1,5), over the BNB's 0.3 at n-gram range (1,4). A better MNB F1-score of 0.62 at n-gram ranges (1,4) and (1,5) over the BNB's 0.24 at n-gram range (1,4). For the 2-Class dataset, MNB achieved its highest accuracy score of 0.9224 at n-gram range (1, 6), over BNB's 0.87795 at n-gram range (1,4). MNB's precision, recall, and f1-score are all at 0.92 at n-gram ranges (1,4), (1,5), and (1,6), whereas BNB's highest precision score of 0.89, highest recall score of 0.88 and highest f1-score of 0.88 all at n-gram range (1,4).

Bernoulli NB (6-Class dataset)										
n-gram range		(1, 4)	(1, 5)	(1, 6)	(2, 4)	(2, 5)	(2, 6)	(3, 4)	(3,5)	(3,6)
Accuracy Score		0.30108	0.29032	0.29032	0.16129	0.16129	0.16129	0.16129	0.13978	0.12903
Precision	caa	0	0	0	0	0	0	0	0	0
	cad	0.36	0.44	0.57	0.6	0.75	1	1	1	1
	domgs	0.17	0.17	0.19	0.11	0.11	0.11	0.1	0.1	0.11
	infs	0	0	0	0	0	0	0	0	0
	notc	0.41	0.38	0.27	0	0	0	0	0	0
	pmfs	0.86	0.83	0.83	0.75	0.75	0.75	0.75	0.67	0.67
	AVE	0.25	0.26	0.26	0.2	0.22	0.26	0.25	0.24	0.24
Recall	caa	0	0	0	0	0	0	0	0	0
	cad	0.31	0.31	0.31	0.23	0.23	0.23	0.23	0.15	0.08
	domgs	1	1	1	1	1	1	1	1	1
	infs	0	0	0	0	0	0	0	0	0
	notc	0.6	0.6	0.6	0	0	0	0	0	0
	pmfs	0.46	0.38	0.38	0.23	0.23	0.23	0.23	0.15	0.15
	AVE	0.3	0.29	0.29	0.16	0.16	0.16	0.16	0.14	0.13
F1-Score	caa	0	0	0	0	0	0	0	0	0
	cad	0.33	0.36	0.4	0.33	0.35	0.38	0.38	0.27	0.14
	domgs	0.29	0.29	0.32	0.2	0.2	0.2	0.19	0.19	0.19
	infs	0	0	0	0	0	0	0	0	0
	notc	0.49	0.46	0.37	0	0	0	0	0	0
	pmfs	0.6	0.53	0.53	0.35	0.35	0.35	0.35	0.25	0.25
	AVE	0.24	0.23	0.22	0.12	0.12	0.12	0.12	0.09	0.07

Table 2: Summary of accuracy scores and classification results for BNB in 6-class dataset

Multinomial NB (6-Class dataset)										
n-gram range		(1, 4)	(1, 5)	(1, 6)	(2, 4)	(2, 5)	(2, 6)	(3, 4)	(3,5)	(3,6)
Accuracy Score		0.63441	0.63441	0.60215	0.43011	0.43011	0.43011	0.25806	0.25806	0.24731
Precision	caa	0.75	0.75	0.67	0.8	0.8	0.8	1	1	1
	cad	0.42	0.42	0.4	0.35	0.35	0.35	0.5	0.5	0.5
	domgs	0.5	0.5	0.5	0.22	0.22	0.22	0.12	0.12	0.12
	infs	0.91	0.91	0.9	0.89	0.89	0.89	0.83	0.83	0.83
	notc	0.86	0.86	0.8	0.71	0.71	0.71	0	0	0
	pmfs	0.67	0.67	0.63	0.5	0.5	0.5	0.8	0.8	0.8
	AVE	0.72	0.72	0.68	0.65	0.65	0.65	0.62	0.62	0.62
Recall	caa	0.27	0.27	0.18	0.18	0.18	0.18	0.14	0.14	0.09
	cad	0.77	0.77	0.77	0.54	0.54	0.54	0.23	0.23	0.23
	domgs	1	1	1	0.89	0.89	0.89	1	1	1
	infs	0.48	0.48	0.43	0.38	0.38	0.38	0.24	0.24	0.24
	notc	0.8	0.8	80	0.33	0.33	0.33	0	0	0
	pmfs	0.92	0.92	0.92	0.62	0.62	0.62	0.31	0.31	0.31
	AVE	0.63	0.63	0.6	0.43	0.43	0.43	0.26	0.26	0.25
F1-Score	caa	0.4	0.4	0.29	0.3	0.3	0.3	0.24	0.24	0.17
	cad	0.54	0.54	0.53	0.42	0.42	0.42	0.32	0.32	0.32
	domgs	0.67	0.67	0.67	0.36	0.36	0.36	0.22	0.22	0.22
	infs	0.62	0.62	0.58	0.53	0.53	0.53	0.37	0.37	0.37
	notc	0.83	0.83	0.8	0.45	0.45	0.45	0	0	0
	pmfs	0.77	0.77	0.75	0.55	0.55	0.55	0.44	0.44	0.44
	AVE	0.62	0.62	0.57	0.43	0.43	0.43	0.27	0.27	0.25

Table 3: Summary of accuracy scores and classification results for MNB in 6-class dataset

Bernoulli NB (2-class dataset)										
n-gram range		(1, 4)	(1, 5)	(1, 6)	(2, 4)	(2, 5)	(2, 6)	(3, 4)	(3, 5)	(3, 6)
Accuracy Score		0.87795	0.8448	0.81905	0.79612	0.7679	0.74956	0.74145	0.71464	0.69771
Precision	not informative	0.81	0.77	0.74	0.71	0.69	0.67	0.66	0.64	0.62
	informative	0.97	0.97	0.97	0.97	0.98	0.97	0.98	0.97	0.97
	AVE	0.89	0.87	0.86	0.84	0.83	0.82	0.82	0.81	0.8
Recall	not informative	0.98	0.98	0.98	0.98	0.99	0.99	0.99	0.99	0.99
	informative	0.78	0.71	0.66	0.61	0.55	0.52	0.5	0.44	0.41
	AVE	0.88	0.84	0.82	0.8	0.77	0.75	0.74	0.71	0.7
F1-Score	not informative	0.89	0.86	0.84	0.83	0.81	0.8	0.79	0.77	0.77
	informative	0.87	0.82	0.78	0.75	0.71	0.67	0.66	0.61	0.58
	AVE	0.88	0.84	0.81	0.79	0.76	0.74	0.73	0.69	0.67

Table 4: Summary of accuracy scores and classification results for BNB in 2-class dataset

Multinomial NB (2-class dataset)										
n-gram range		(1, 4)	(1, 5)	(1, 6)	(2, 4)	(2, 5)	(2, 6)	(3, 4)	(3, 5)	(3, 6)
Accuracy Score		0.921	0.92205	0.9224	0.84974	0.84868	0.84797	0.61481	0.61446	0.61481
Precision	not informative	0.96	0.96	0.95	0.93	0.93	0.93	0.87	0.87	0.87
	informative	0.89	0.89	0.9	0.79	0.79	0.79	0.57	0.57	0.57
	AVE	0.92	0.92	0.92	0.86	0.86	0.86	0.72	0.7	0.72
Recall	not informative	0.88	0.88	0.89	0.75	0.75	0.75	0.26	0.27	0.27
	informative	0.96	0.96	0.96	0.95	0.95	0.94	0.96	0.96	0.96
	AVE	0.92	0.92	0.92	0.85	0.85	0.85	0.61	0.61	0.61
F1-Score	not informative	0.92	0.92	0.92	0.83	0.83	0.83	0.41	0.41	0.41
	informative	0.92	0.93	0.93	0.86	0.86	0.86	0.72	0.71	0.71
	AVE	0.92	0.92	0.92	0.85	0.85	0.85	0.56	0.56	0.56

Table 5: Summary of accuracy scores and classification results for MNB in 2-class dataset

Comparing the two estimators' accuracy scores and classification results, Multinomial Naïve Bayes performed better than the Bernoulli Naïve Bayes for both 2-Class datasets and 6-Class datasets.

A script was also made to generate a visual representation of the confusion matrices for both Bernoulli and Multinomial Naïve Bayes classifiers. Confusion matrices are used to visualize how many of the test data sets' tweets were classified as True Positive (TP), True Negative (TN), False Positive (FP) and False Negative (FN).

For the 6-class dataset, the compared confusion matrices for BNB and MNB are from the n-gram range of (1,4) as the highest accuracy scores for both estimators were achieved at this range.

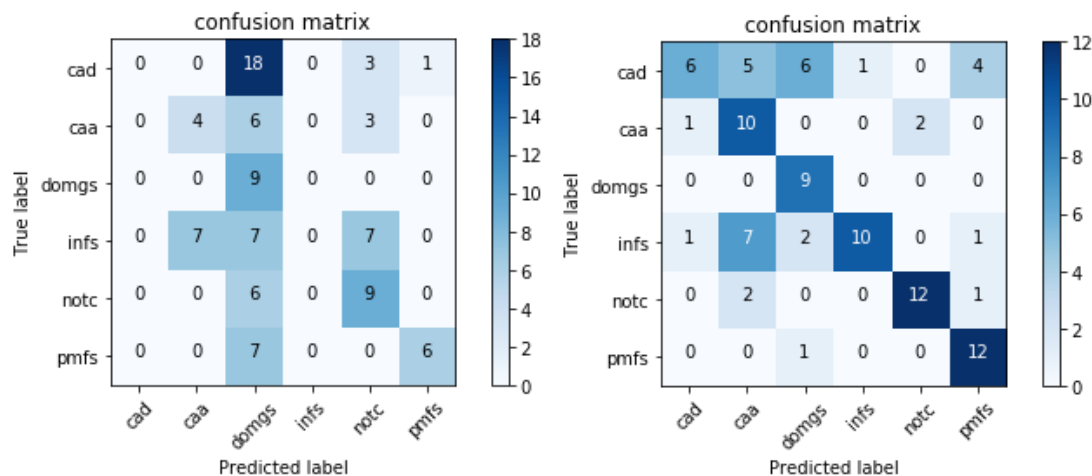


Figure 21: Confusion Matrices of BNB and MNB at range (1,4) for the 6-Class dataset

The confusion matrix of BNB shows that there were 4 out of 13 instances of the caution and advice (CAA) tweets, which were correctly labeled. All 9 instances of the donations of money goods services (DOMGS) tweets were correctly labeled. 9 out of 15 instances of the none of the choices (NOTC) tweets were correctly labeled. 6 out of 13 instances of the people missing found seen (PMFS) tweets were correctly labeled. No instances of caution and advice (cad) and information source (INFS) were correctly labeled.

The confusion matrix of MNB shows that there were 6 out of 22 instances of the casualties and damage (CAD) tweets were correctly labeled. 10 out of the 13 instances of caution and advice (CAA) tweets were correctly labeled. All 9 instances of the donations of money goods services (DOMGS) tweets were correctly labeled. 10 out of 21 instances of the information source (INFS) tweets were correctly labeled. 12 out of 15 instances of the none of the choices (NOTC)

tweets were correctly labeled, and 12 out of 13 instances of the people missing found seen (PMFS) tweets were correctly labeled.

For the 2-class dataset, the compared confusion matrices for BNB and MNB are from the n-gram range of (1,4) and (1,6) as the highest accuracy score for the BNB estimators was achieved at n-gram range (1,4) and the highest accuracy score for the MNB estimator was achieved at n-gram range (1,6).

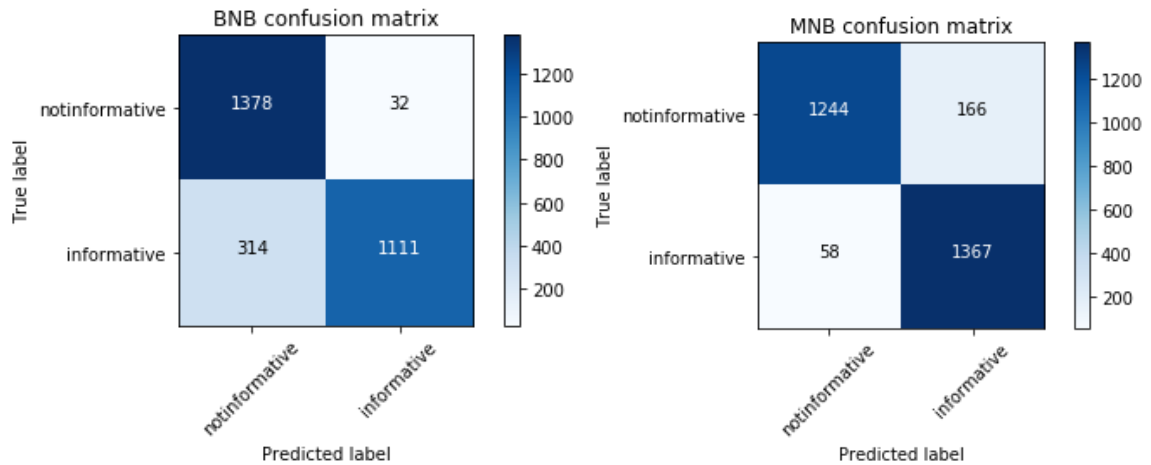


Figure 22: Confusion Matrices of BNB and MNB at range (1,4) for the 2-Class dataset

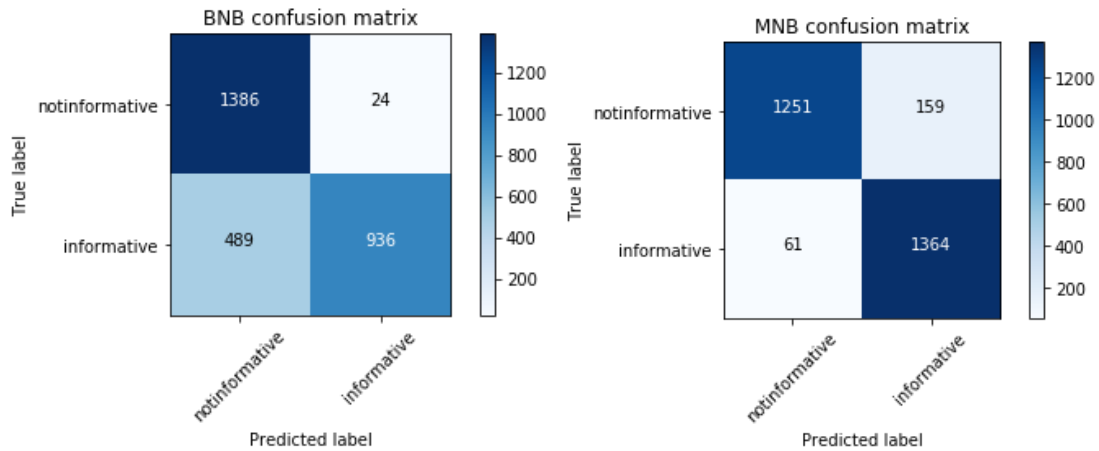


Figure 23: Confusion Matrices of BNB and MNB at range (1,6) for the 2-Class dataset

For the n-gram range (1,4) confusion matrices, BNB's confusion matrix shows that 1378, out of the 1410 instances of non-informative tweets, and 1111 out of the 1425 instances of informative tweets were correctly labeled. The confusion matrix of MNB shows that there were 1244 out of the 1410 instances of non-

informative tweets that were correctly labeled, with 1367 out of the 1425 instances of informative tweets that were correctly labeled.

For the n-gram range (1,6) confusion matrices, BNB's confusion matrix shows that 1386, out of the 1410 instances of non-informative tweets, and 936 out of the 1425 instances of informative tweets were correctly labeled. The confusion matrix of MNB shows that there were 1255 out of the 1410 instances of non-informative tweets that were correctly labeled, with 1364 out of the 1425 instances of informative tweets that were correctly labeled.

A script was also made to compute and generate the respective AUC-ROC diagrams to determine between n-gram range (1,4) and (1,6) performed better when classifying tweets using Multinomial Naïve Bayes. An AUC-ROC diagram is a performance measurement score intended for classification models. The higher an AUC score implies that a classification model is better at distinguishing between classes. The best classification model has an AUC score of 1.0 whereas an inefficient classifier score at 0.5 or even less. Receiver Operating Characteristics (ROC) is the curve plotted on a cartesian plane, with False Positive Rates (FPR) or *specificity* on the X-axis, and True Positive Rates (TPR) or *recall* on the Y-axis. Each point on the ROC curve represents an FPR-TPR point that corresponds to a decision threshold. ROC curves that are leaning towards the upper left corner of the graph indicates a higher accuracy of determining TPRs over FPRs.

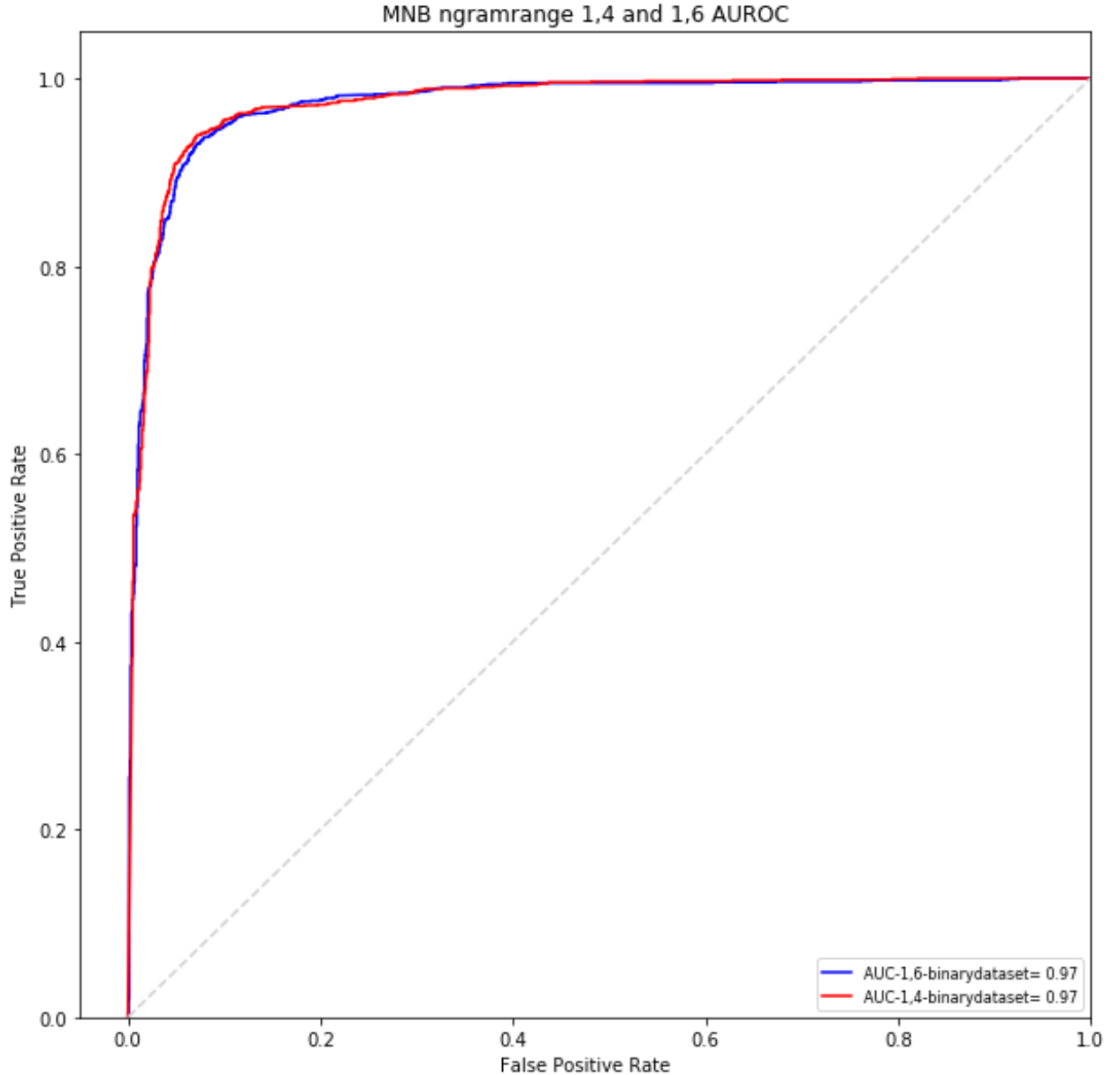


Figure 24: AUC-ROC curve and AUC score for both MNB models

The AUC score for both models MNB n-gram range (1,6) (blue) and MNB n-gram range (1,4) (red) models are at 0.97, which is indicative of both models' efficiency in determining informative and not informative tweets apart.

5.2 Implementing the Web Application

The output of implementing the Web Application is a visualized and interactive topic map of the various topics found in the tweet_text data. The following three

subprocesses were done to implement the Web Application: (1) Exporting result data, (2) Creating the JSON file, (3) Implementing the Visualization of results in Django.

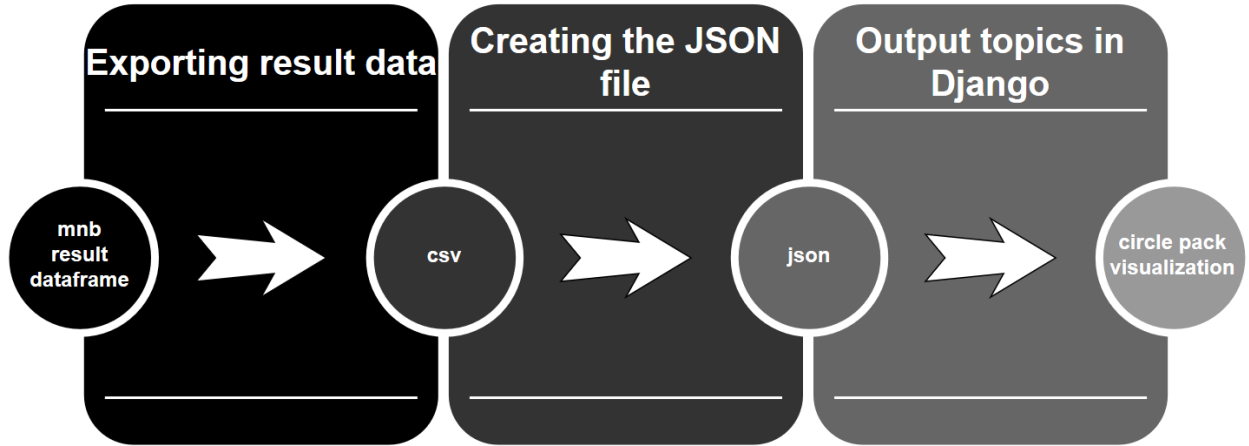


Figure 25: Data flow from classification to visualization

5.2.1 Exporting result data

After the classification test results were generated, the results of the Multinomial Naïve Bayes pipeline proved to have better accuracy, precision, recall, and F1-scores for both 6-class and 2-class datasets. For the 6-class dataset, the test data that was needed to be visualized were the data from the n-gram range (1, 4). For the 2-class dataset, the test data that was needed to be visualized were the data from the n-gram range (1, 6). These were n-gram ranges chosen because the highest accuracy was attained from their corresponding class datasets. The following columns from the test data were assigned into respective pandas data frames: `tweet_text`, `y_pred`, and `y_test`. The `y_pred` column contained MNB's predicted labels for the testing `tweet_data`. The `y_test` column contained the ground truth labels from the manual annotation.

```
mnb_df_bin = pd.DataFrame({'tweet_text': x_test.tolist(),
                           'y_pred': y_pred.tolist(),
                           'y_test': y_test.tolist()})
mnb_df = pd.DataFrame({'tweet_text': x_test.tolist()})
mnb_df['predicted_inf_class'] = mnb_df['labelled_inf_class'] = ""
cnt1 = cnt0 = 0
for i, x in mnb_df_bin.iterrows():
    mnb_df.at[i, 'predicted_inf_class'] = x['y_pred']
    mnb_df.at[i, 'labelled_inf_class'] = x['y_test']

new_df = df1.merge(mnb_df, how="inner", left_on="tweet_text", right_on="tweet_text")
new_df.sort_values(by="tweet_text")
```

Figure 26: Script to export result n-gram range (1,4) data

The values of the `y_test` column from the 2-class dataset columns were binarized into 0 and 1 values during feature extraction and had to be relabeled to not informative (for 0 values) and informative (for 1 values). The following script was made to do this:

```
mnb_df_bin = pd.DataFrame({'tweet_text': x_test.tolist(),
                           'y_pred': y_pred.tolist(),
                           'y_test': y_test.tolist()})
mnb_df = pd.DataFrame({'tweet_text': x_test.tolist()})
mnb_df['predicted_inf_class'] = mnb_df['labelled_inf_class'] = ""
cnt1 = cnt0 = 0
for i, x in mnb_df_bin.iterrows():
    if x['y_pred'] == 1:
        mnb_df.at[i, 'predicted_inf_class'] = "informative"
    else:
        mnb_df.at[i, 'predicted_inf_class'] = "notinformative"

for i, x in mnb_df_bin.iterrows():
    if x['y_test'] == [1]:
        mnb_df.at[i, 'labelled_inf_class'] = "informative"
    else:
        mnb_df.at[i, 'labelled_inf_class'] = "notinformative"

new_df = df1.merge(mnb_df, how="inner", left_on="tweet_text", right_on="tweet_text")
new_df.sort_values(by='tweet_text')
```

Figure 27: Script to export and relabel result *n*-gram range (1,6) data

After the binarized `y_test` and `y_pred` data were relabeled, these were merged into the data frame containing the CSV data (*df1*) using pandas data frame's *merge* function. Since *df1* contained the entire dataset (training and testing datasets), an intersection of keys from both *df1* and *mnb_df* data frames was needed to be done to associate which rows belonged to one another. The pandas *.merge()* function made this possible. This function merges two data frames by performing database-style join operations by columns or by indexes. The *.merge()* function accepts the following four parameters: *mnb_df*, *how="inner"*, *left_on="tweet_text"*, *right_on="tweet_text"*.

The *mnb_df* variable stored the Pandas data frame containing the columns: *tweet_text*, *predicted_inf_class*, and *labelled_inf_class*. The *predicted_inf_class* and the *labelled_inf_class* columns contained either informative or not informative values of the *y_pred* and *y_test* columns.

The *how="inner"* parameter is a pythonic key-word argument, which states that an intersection of keys from both frames. The *left_on="tweet_text"* pythonic key-word argument states that the column name to join on in the left data frame (*df1*), is *tweet_text*. The *right_on="tweet_text"* pythonic key-word

argument states that the column name to join on in the right data frame (mnbf_df) is tweet_text.

```
new_cleaned_df = new_df.drop_duplicates(subset=['tweet_text'])
```

Figure 28: Script to remove duplicates in the intersection of data frames

The results of the merge operation were stored in the new_df variable, which was in turn, appended with the .drop_duplicates() method. The .drop_duplicates() method is a pandas data frame method which eliminated any duplicate rows in a data frame. Results of this operation were stored in the new_cleaned_df variable.

```
new_cleaned_df['agreed_inf_class'] = ""
count = 0

for i,x in new_cleaned_df.iterrows():
    if new_cleaned_df.at[i,'predicted_inf_class'] == new_cleaned_df.at[i,'labelled_inf_class']:
        new_cleaned_df.at[i,'agreed_inf_class'] = new_cleaned_df.at[i,'predicted_inf_class']
    else:
        new_cleaned_df = new_cleaned_df.drop([i])
```

Figure 29: Script to omit any conflicting predicted and manually classified labels

The new_cleaned_df variable was then appended with a new column, agreed_inf_class. This column identifies which among the tweets had the same predicted and manually classified labels. If the tweet or data frame row had conflicting predicted and manually classified labels, these were discarded using the .drop() method.

```
new_cleaned_df.to_csv("ready_for_visualization.csv")
```

Figure 30: Script to save the data frame into a CSV file

After this process, the new_cleaned_df data frame was exported as a CSV file named "ready_for_visualization.csv" for the 2-class dataset.

For the 6-class dataset, similar processes were done to export its result data. A separate CSV file named "6class_visualization.csv" was also exported for the 6-class dataset.

5.2.2 Creating the JSON file

Before any visualization was done, a JSON file was needed to serve as the basis for the Circle Packing visualization to be rendered. This JSON file contains the following node values: (1) the parent class, (2) child class(es), and, (3) corresponding grand-child classes.

For the 2-class dataset, the parent classes for this implementation are *informative* and *not informative* tweet labels. The child classes for the informative tweet label are the following sub-classes: *information source*, *donations of money goods services*, *people missing found seen*, *caution and advice*, and, *casualties and damage*. The lone child class for not informative tweet label is the sub-class, *none of the choices*. Each of these child classes has their respective 1-gram to 6-grams. These n-gram clusters contain the common 10 n-grams per n-gram child class. For the 6-class dataset, the parent classes are the sub-classes of informative and not-informative, and their child classes are their respective n-grams.

A script was made to identify and store each tweet text's 1-gram to 6-grams in respective columns in the `ready_for_visualization.csv` file. A for-loop was utilized to iterate through the rows of the generated CSV. Python NLTK's `ngrams()` method was used to generate n-grams for each tweet text. The `ngrams()` method accepted two arguments: a pythonic sequence, and the number of n-grams to be created. A variable for each n-gram count was assigned the `ngrams()` method to generate the n-gram values identified for each tweet text.

```
grams1 = ngrams(row['tweet_text'].split(),1)
grams2 = ngrams(row['tweet_text'].split(),2)
grams3 = ngrams(row['tweet_text'].split(),3)
grams4 = ngrams(row['tweet_text'].split(),4)
grams5 = ngrams(row['tweet_text'].split(),5)
grams6 = ngrams(row['tweet_text'].split(),6)
```

Figure 31: Script to identify 1-gram to 6-gram terms

Each of these n-grams is separated by the ';' delimiter and are all saved on their respective n-gram column. Once all of the tweet texts had their respective n-gram rows populated, these were saved into the `ready_for_visualization.csv` along their corresponding `tweet_text` and `row`. The tweet below contains the various n-gram term combinations (1,6) for the tweet “Oriental Mindoro PDRRMO At least 4 dead 20000 families or 87000 persons evacuated due to NonaPH via rongagalac”.

1_grams_cad	2_grams_cad	3_grams_cad
Oriental;Mindoro;PDRRMO;Atleast;4;d ead;20000;families;or;87000;persons;e vacuated;due;to;NonaPH;via;rongagal ac	Oriental Mindoro;Mindoro PDRRMO;PDRRMO Atleast;Atleast 4;4 dead;dead 20000;20000 families;families or;or 87000;87000 persons;persons evacuated;evacuated due;due to;to NonaPH;NonaPH via;via rongagalac	Oriental Mindoro PDRRMO;Mindoro PDRRMO Atleast;PDRRMO Atleast 4;Atleast 4 dead;4 dead 20000;dead 20000 families;20000 families or;families or 87000;or 87000 persons;87000 persons evacuated;persons evacuated due;evacuated due to;due to NonaPH;to NonaPH via;NonaPH via rongagalac
4_grams_cad	5_grams_cad	6_grams_cad
Oriental Mindoro PDRRMO Atleast;Mindoro PDRRMO Atleast 4;PDRRMO Atleast 4 dead;Atleast 4 dead 20000;4 dead 20000 families;dead 20000 families or;20000 families or 87000;families or 87000 persons;or 87000 persons evacuated;87000 persons evacuated due;persons evacuated due to;evacuated due to NonaPH;due to NonaPH via;to NonaPH via rongagalac	Oriental Mindoro PDRRMO Atleast 4;Mindoro PDRRMO Atleast 4 dead;PDRRMO Atleast 4 dead 20000;Atleast 4 dead 20000 families;4 dead 20000 families or;dead 20000 families or 87000;20000 families or 87000 persons;families or 87000 persons evacuated;or 87000 persons evacuated due;87000 persons evacuated due to;persons evacuated due to NonaPH;evacuated due to NonaPH via;due to NonaPH via rongagalac	Oriental Mindoro PDRRMO Atleast 4 dead;Mindoro PDRRMO Atleast 4 dead 20000;PDRRMO Atleast 4 dead 20000 families;Atleast 4 dead 20000 families or;4 dead 20000 families or 87000;dead 20000 families or 87000 persons;20000 families or 87000 persons evacuated;families or 87000 persons evacuated due;or 87000 persons evacuated due to;87000 persons evacuated due to NonaPH;persons evacuated due to NonaPH via;evacuated due to NonaPH via rongagalac

Table 6: n-gram range (1,6) for tweet text

The data from the updated CSV file was used to generate a JSON file which was needed for the d3.js library to render a visualization of the topics. The JSON file based its structure on the following diagrams:

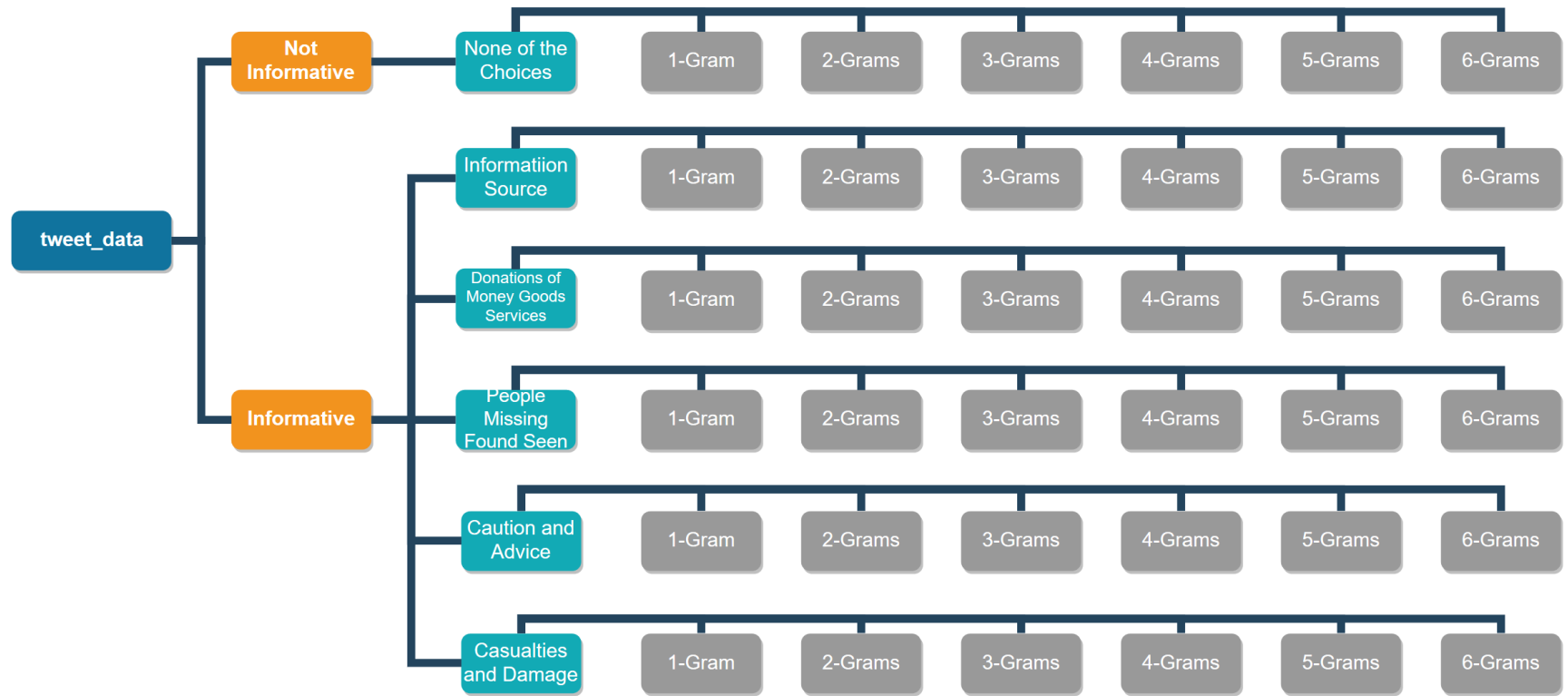


Figure 32: Hierarchy of classes and corresponding child classes for the 2-class dataset

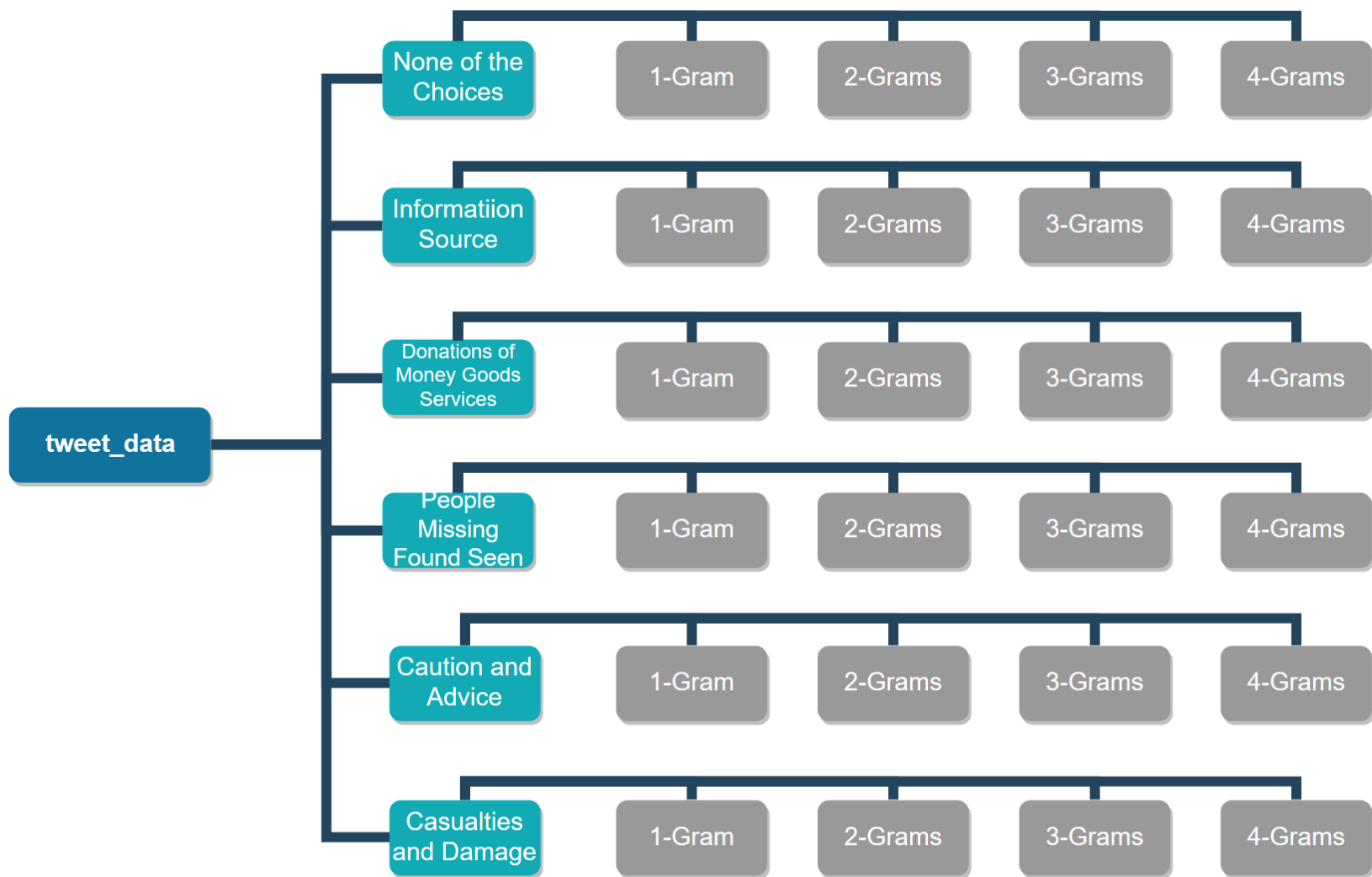


Figure 33: Hierarchy of classes and corresponding child classes for the 6-class dataset

5.2.3 Implementing the Visualization of Results in Django

Django follows a Model-View-Template architecture. For this project, Django version 2.1 was used and installed in a virtual environment set up on a LUbuntu machine. The Django framework relies on five python files which are generated automatically once a Django project and application is initialized; these are (1) settings.py, (2) urls.py, (3) models.py, (4) views.py and (5) the HTML files found in the templates directory.

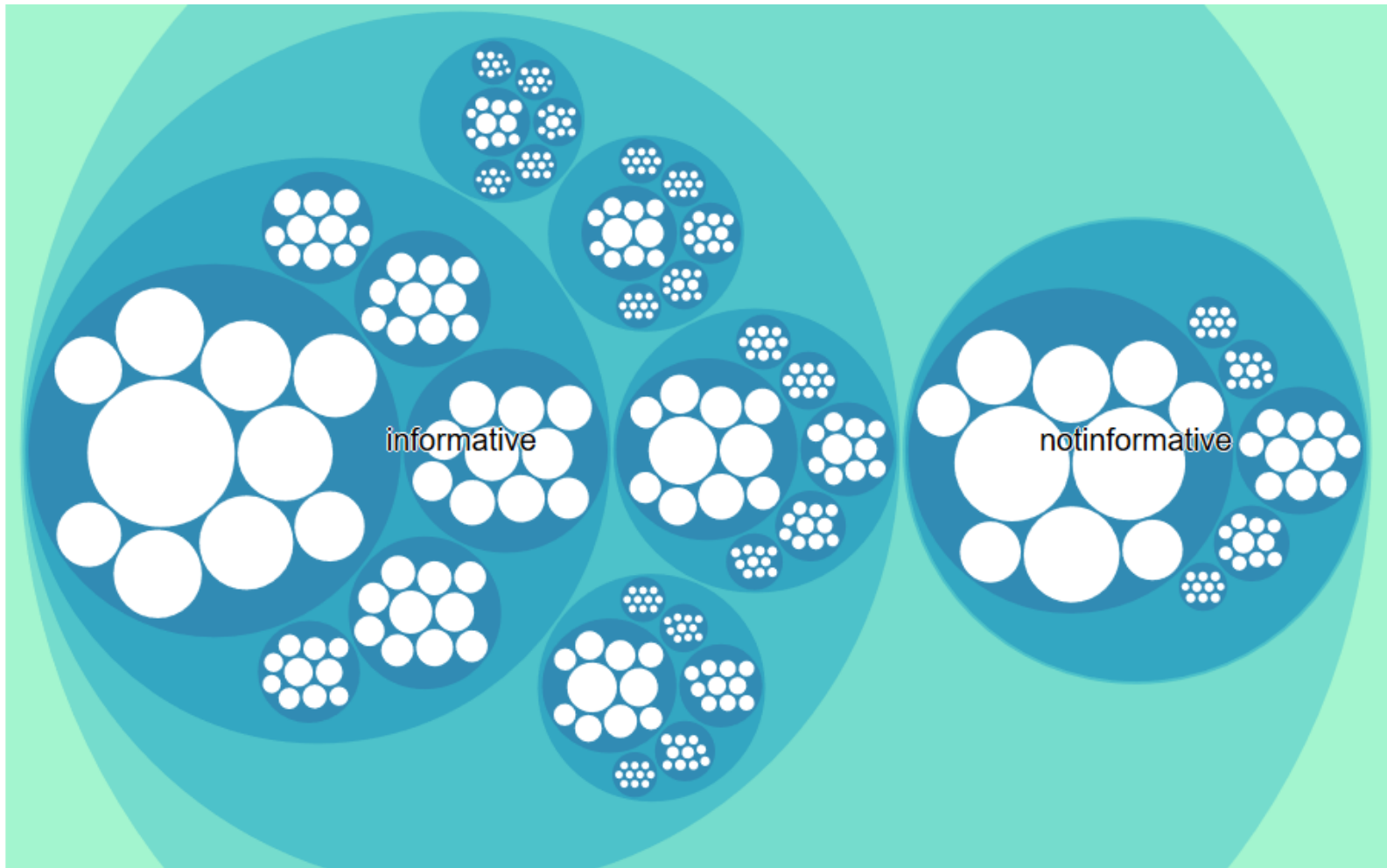
The settings.py file contains the configurations of the Django installation such as, what applications are to be included in the web app, what is the time zone to be followed, what are the default directory paths for static and media files, etc. The urls.py file contains the URL configuration, which maps out URL path expressions to Python functions, which are found in the views.py file of an application.

The models.py file contains the source of information of the data being used in the Django web application, such as the data types of these data, which data are essential, and what are the behaviors of these data. For this project, the models.py file wasn't used, as the data being visualized on the web page are all from the static output.json file generated inside the views.py.

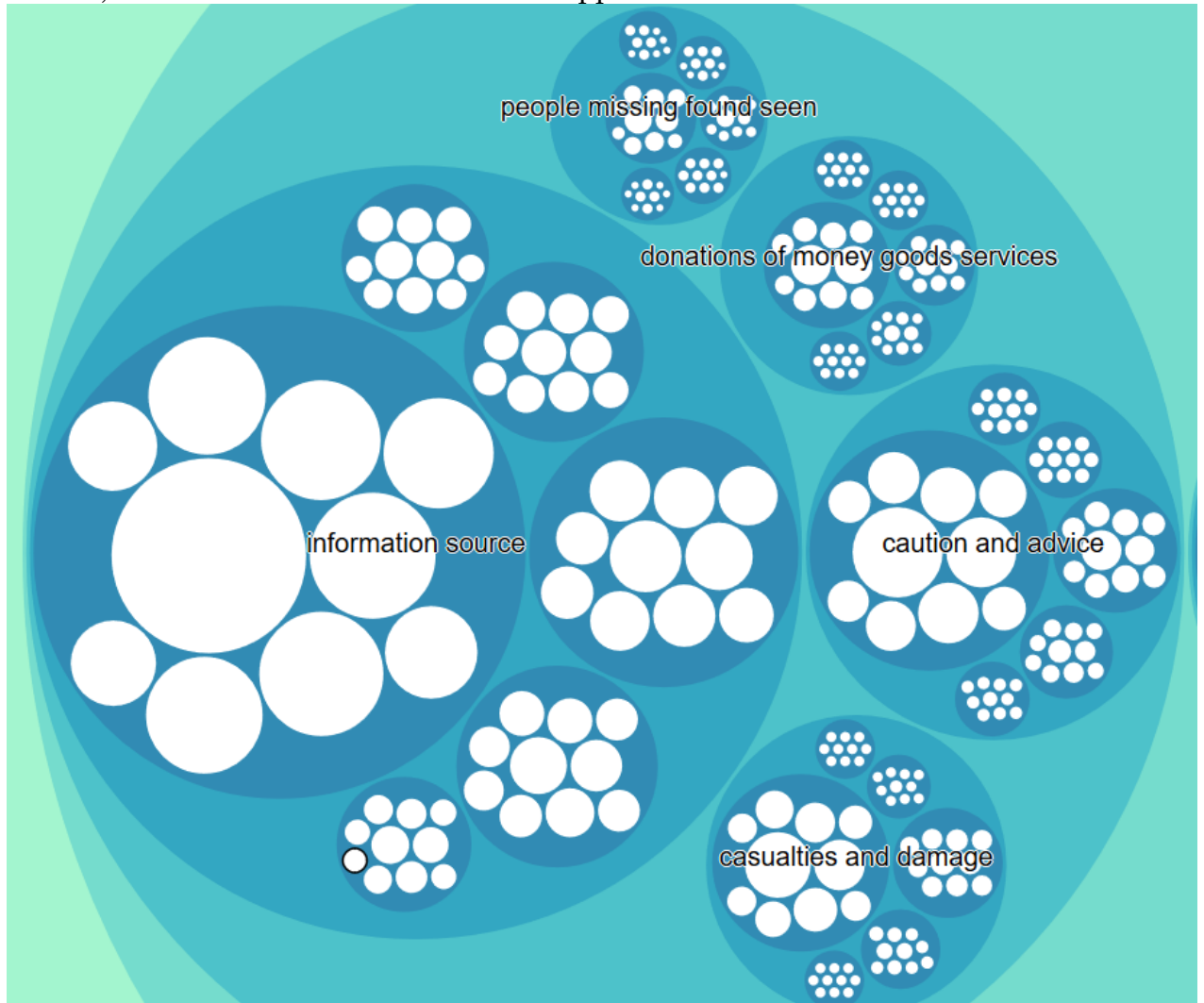
The views.py file contains all Python user-defined functions which act as the *controllers* of data. This file is where computations and other data are passed to certain template or HTML files. This is also where the data is molded from the ready_for_visualization.csv into the output.json file. The HTML files found in the templates directory is where users view dynamic and static data.

Visualization of the n-gram features was implemented using the D3js library's circle packing tool. D3.js is a Javascript library which allows beautiful and interactive visualizations of statistical data, and classes which are stored in JSON file. A premade Circle Packing implementation was customized and used to visualize the data in the JSON file (Bostock, n.d.). Each node in the JSON file was visualized through these circle shapes. On page load, corresponding circle-packing clusters would be generated based on the size of each class' n-gram population.

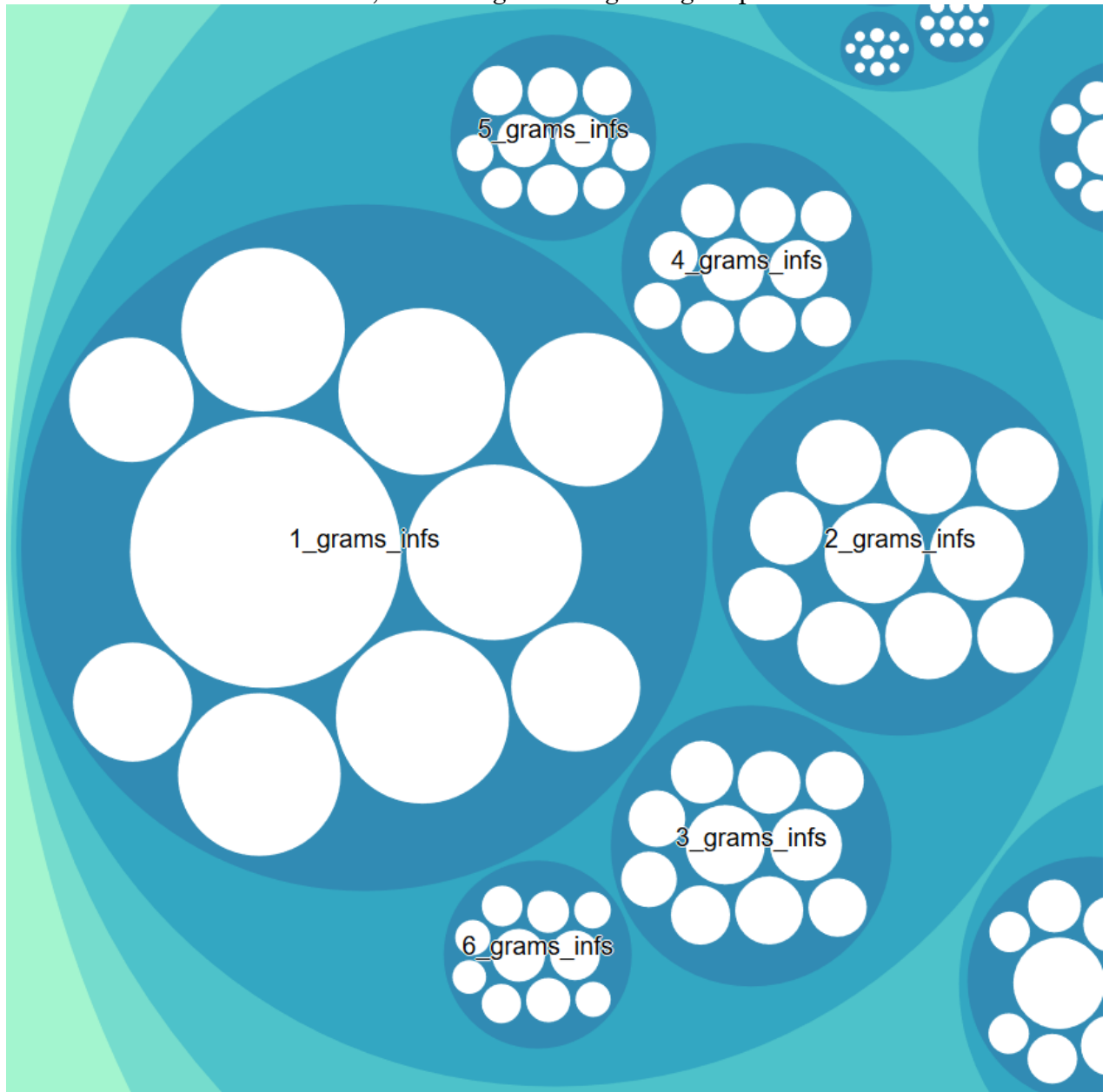
This is the default view for the 2-class dataset.



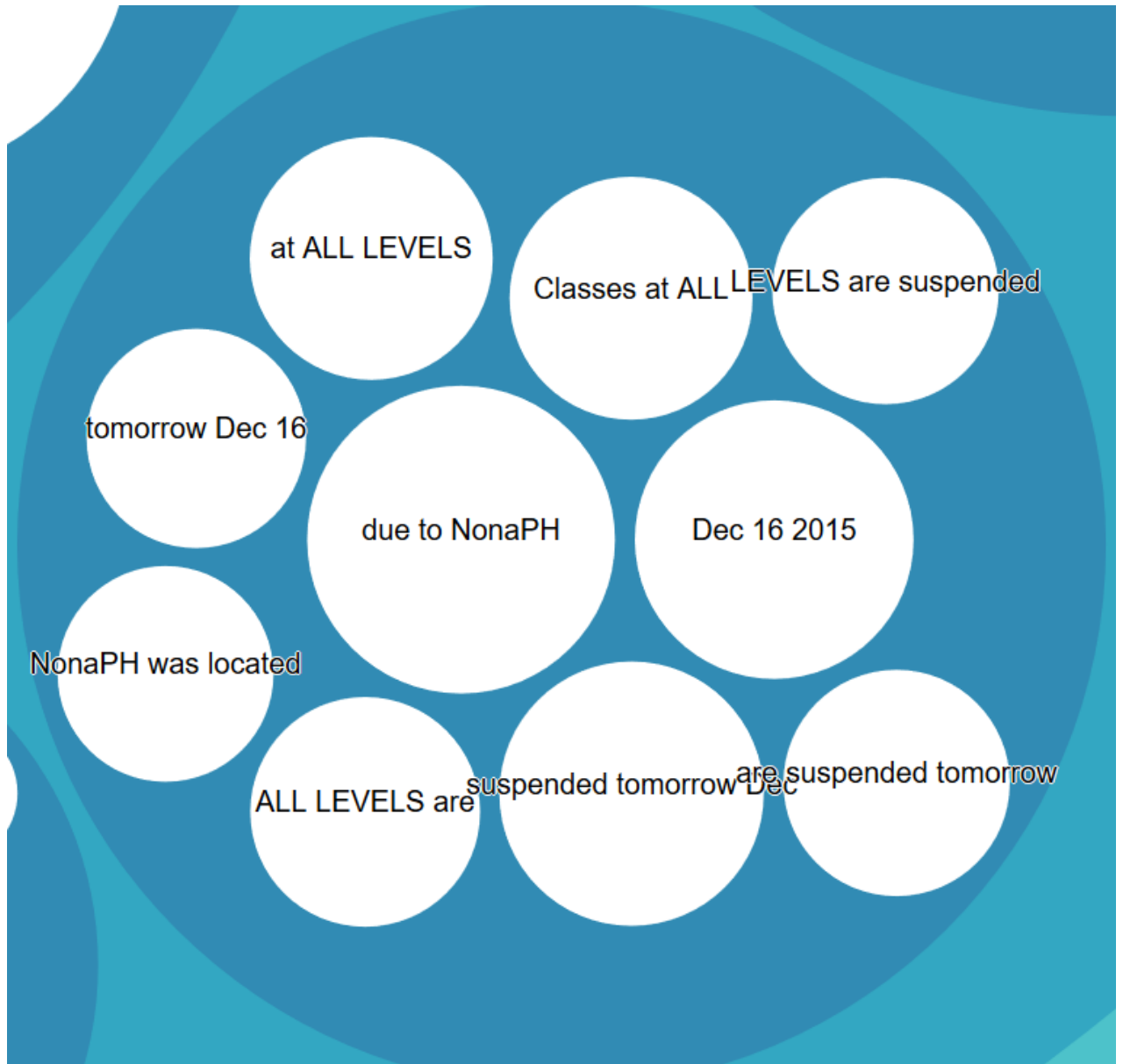
Clicking on the “informative” circle would zoom into the contents of the former, and labels of its children would appear.



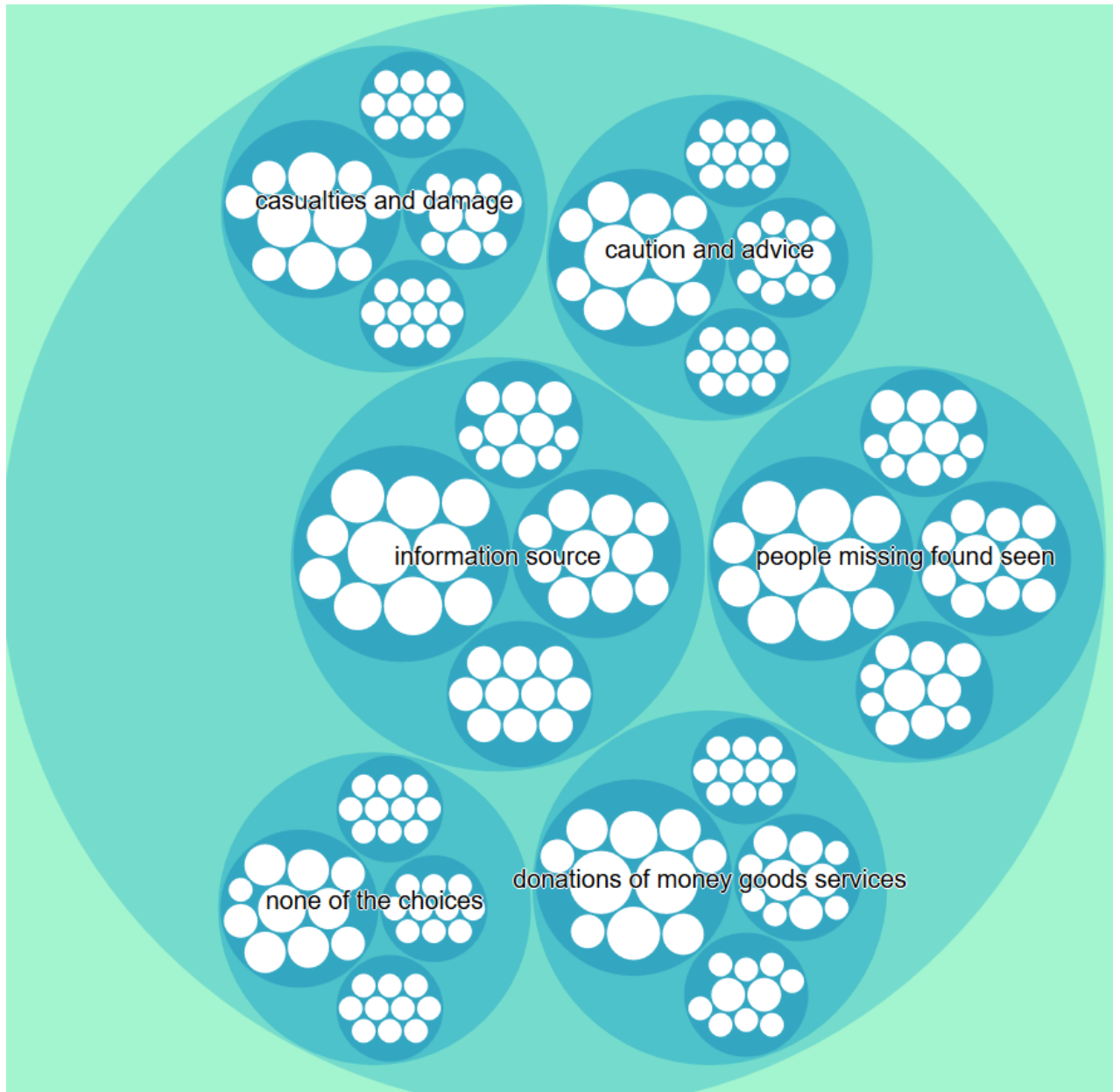
Clicking on the “information source” (a subclass/child of the informative class) would zoom into its contents, revealing its 6 n-gram groups.



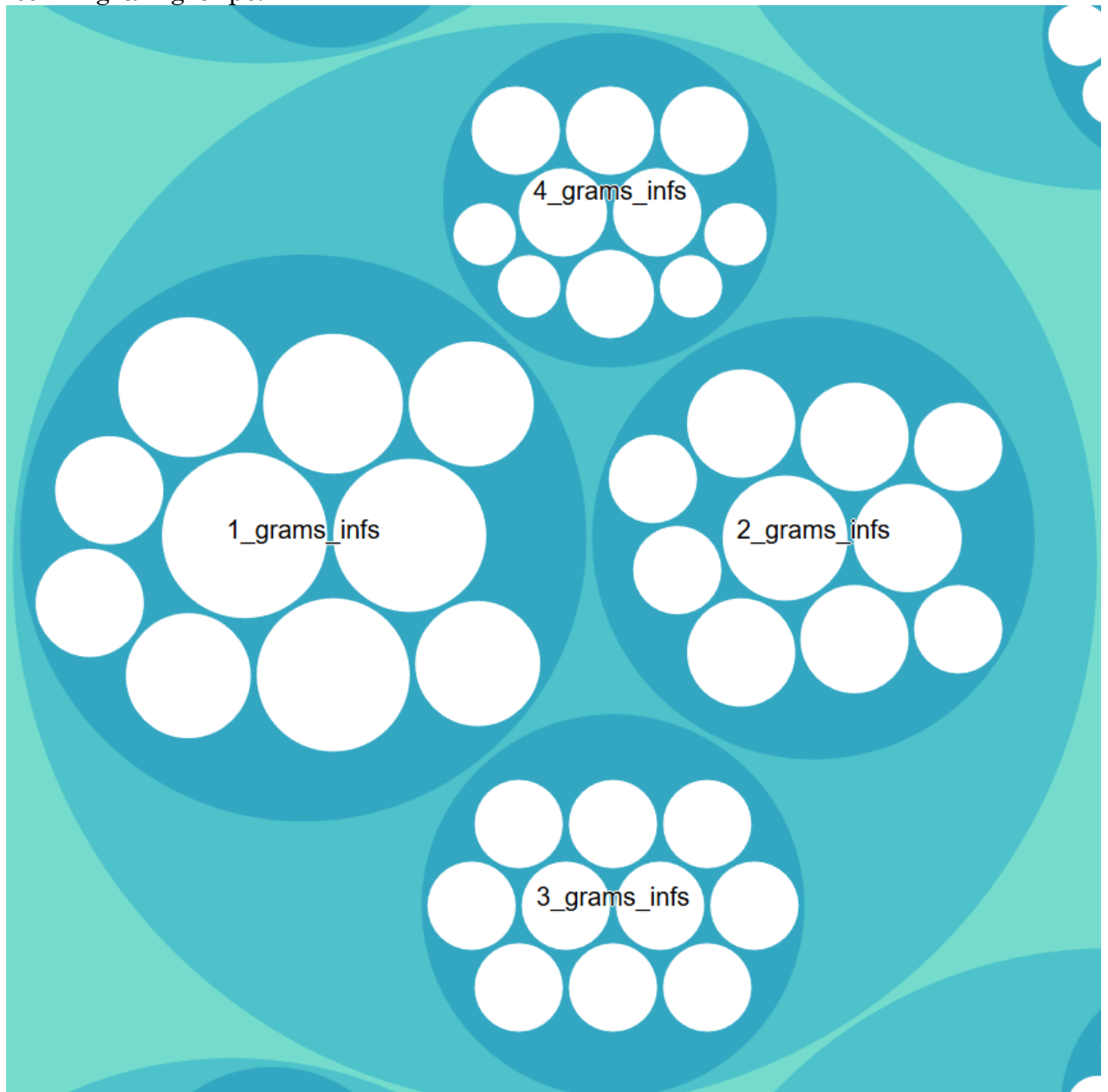
Clicking on one of the n-gram groups (3-grams) would zoom into its contents, revealing 10 of the common instances of the n-gram cluster of words.



This is the default view for the 6-class dataset.



Clicking on the “information source” would zoom into its contents, revealing its 4 n-gram groups.



Clicking on one of the n-gram groups (3-grams) would zoom into its contents, revealing 10 of the common instances of the n-gram cluster of words.



CHAPTER 6

CONCLUSIONS AND RECOMMENDATIONS

6.1 Conclusion

The typhoon Nona dataset was gathered successfully using Twitter’s streaming API – through the pythonic library Tweepy. Tweepy allowed Twitter interactions particularly searching for tweets with the hashtag #nonaPH. However, the API could not do *data mining*. Data mining is the process of analyzing the patterns of large data sets by using data cleaning, natural language processing, and machine learning methods. The API is limited to giving access to twitter interactions, such as tweeting (creating and posting tweets), deleting personal tweets, and searching for tweets based on a text query or category (hashtag, user search, or keyword search).

The necessary tools needed to analyze the tweet corpora consists of developed scripts, and, methods belonging to the Sklearn and NLTK Python libraries. Before tweet classification, the tweet corpora had undergone various pre-processing procedures: removal of symbols, hyperlinks, user mentions, hashtag symbols, blank cells, duplicate tweets, retweeted tweets, and stop words. Each of the tweets was also subject to CountVectorization and TfIDF transformation – which made the data ‘understandable’ to the chosen Bayesian classifiers.

The validity of a tweet in the context of Typhoon disasters focused on its informative classes. These classes are (1) Caution and advice, (2) Casualties and damage, (3) Donations of money, goods or services, (4) People missing, found or seen, and (5) Information sources. These five classes served as both criteria and factors for tweet context validity and were the basis behind the Informative label in the Bayesian classifiers.

The tweet classification process was made possible by utilizing Sklearn’s Multinomial and Bernoulli Naïve Bayes methods and was simplified further with the use of Sklearn’s Pipeline method. Between the two Bayesian classifiers, Multinomial Naïve Bayes performed with a better accuracy and classification scores as compared to Bernoulli Naïve Bayes because MNB classifies a tweet text based on the counts it finds of multiple n-grams or term frequency. MNB investigates the importance of each n-gram or term for each tweet text. BNB, on the other hand, bases its classification on the existence of certain n-grams features, in other words, BNB does a checklist of n-grams that

are observed to be present and absent in tweet texts that are labeled with a certain class.

Post-processing is defined to be the process where the results of a classification problem can further be *evaluated* and *visualized* to further the understanding of a given data set (Bruha & Famili, 2000). The post-processing of tweets was done in two parts of the study: (1) During evaluation of the two Bayesian methods and (2) during the visualization of the common n-grams per tweet category.

The results proved that tweet relevance, or its informativeness, in the context of a Typhoon's landfall, can be identified by utilizing the Bayesian variant, Multinomial Naïve Bayes. Among the two variants used, Multinomial Naïve Bayes performed exceptionally better than Bernoulli Naïve Bayes. However, not all tweet features that were gathered was used in the process, because of the nature of the Bayesian classifiers. These classifiers only considered text data and its respective class-labels.

6.2 Recommendations

The following recommendations are suggested based on the drawn conclusions:

1. Consider other tweet features, not only tweet text data in tweet classification problems.
2. Consider the presence of emojis, as these would also prove to be a basis for text sentiments on online platforms.
3. One of the struggles of dealing with classification problems is dealing with imbalanced data. The class count of each informative subclass was very imbalanced. Future researchers may consider data over sampling, under sampling, or gathering more data to address this concern.

REFERENCES

- Arcilla, A., Amoguis, L., & Tan, J. (2013). Retrieved from Emotion Analysis Using Knowledge-Based Neural Networks and Emotional Keyword Dictionary on Detecting Depression: <https://sites.google.com/a/dcs.upd.edu.ph/csp-proceedings/pcsc2013>
- Bostock, M. (n.d.). *Zoomable Circle Packing*. Retrieved from blockbuilder.org: <https://blockbuilder.org/mbostock/7607535>
- Bradley, A., Wall, I., Von Delft, F., Green, D., Deanne, C., & Marsden, B. (2015). *WONKA: Objective novel Complex Analysis for Ensembles of Protein-Ligand Structures*. Retrieved from www.researchgate.net: https://www.researchgate.net/publication/282046466_WONKA_objective_novel_complex_analysis_for_ensembles_of_protein-ligand_structures
- Brooks-Barlett, J. (2018). *Probability Concepts explained: Maximum Likelihood Estimation*. Retrieved from towardsdatascience.com: <https://towardsdatascience.com/probability-concepts-explained-maximum-likelihood-estimation-c7b4342fdbb1>
- Bruha, I., & Famili, A. (2000). *Postprocessing in Machine Learning and Data Mining*. Retrieved from www.kdd.org: www.kdd.org/exploration_files/KDD2000PostWkshp.pdf
- Carreos, J., & Liboon, P. (2014). Retrieved from Identifying Emotions in Tagalog Song Lyrics.
- Castillo, C., Mendoza, M., & Poblete, B. (2010). Retrieved from Twitter Under Crisis: Can We Trust What We RT?: <http://dl.acm.org/citation.cfm?id=1964869>
- Cobo, A., Parra, D., & Navon, J. (2013). Retrieved from Identifying Relevant Messages in a Twitter-based Citizen Channel for Natural Disaster Situations: http://www.researchgate.net/publication/273787950_Identifying_Relevant_Messages_in_a_Twitter-based_Citizen_Channel_for_Natural_Disaster_Situations
- Data-Driven Documents*. (2018). Retrieved from d3js.org: <https://d3js.org>
- De Leon, M., Estuar, R., Lee, J., & Ybañez, M. (2013). Retrieved from Understanding the Behavior of Filipino Twitter users during Disaster: <http://www.globalsciencejournals.com/article/10.7603%2Fs40601-013-0007-z>
- Deanne, C., Wall, I., Green, D., Marsden, B., & Bradley, A. (2014). *OOMMPPAA: A tool to aid directed synthesis by the Combined Analysis of Activity and Structural Data*. Retrieved from ncbi.nlm.nih.gov: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC5349440/>
- Django - Overview*. (2018). Retrieved from www.tutorialspoint.com: https://www.tutorialspoint.com/django/django_overview.htm
- Efron, M. (2010). Hashtag retrieval in a microblogging environment. *Proceedings of the 33rd international ACM SIGIR conference on research and development in information retrieval.*, 787-788.
- Hashtag definition by Merriam-Webster*. (2015). Retrieved from Merriam-Webster.com: <http://www.merriam-webster.com/dictionary/hashtag>

- Imran, M., Elbassuoni, S., Castillo, C., Diaz, F., & Meier, P. (2013). Retrieved from Extracting Information Nuggets from Disaster-Related Messages in Social Media: <http://qcri.org.qa/app/media/1843>
- Kumar, P. (2017). *An Introduction to N-Grams: What are they and why do we need them?* Retrieved from blog.xrds.acm.org: <https://blog.xrds.acm.org/2017/10/introduction-n-grams-need/>
- Landis, R., & Koch, G. (1977). The Measurement of Observer Agreement for Categorical Data. *Biometrics*, Vol. 33, 154-174.
- Naaman, M., Boase, J., & Lai, C. (2010). Retrieved from Is it Really About Me? Message Content in Social Awareness Streams: <http://dl.acm.org/citation.cfm?id=1718953>
- Novalic, A. (2013). *Introduction to Tweepy, Twitter for Python*. Retrieved from Pythoncentral.io: <https://www.pythoncentral.io/introduction-to-tweepy-twitter-for-python/>
- Pittenturf, C. (n.d.). *What is Data Visualization and Why is it Important?* Retrieved from data-visualization.cioreview.com: <https://data-visualization.cioreview.com/cxinsight/what-is-data-visualization-and-why-is-it-important-nid-11806-cid-163.html>
- Radich, R. (2017). *Big Data for Humans: The Importance of Data Visualization*. Retrieved from dataconomy.com: <https://dataconomy.com/2017/05/big-data-data-visualization/>
- Salarpour, A., Bamneshin, M., & Proios, D. (2014). *Sentiment Analysis and Visualization of Social Media Data*. Retrieved from www.researchgate.net: https://www.researchgate.net/publication/268503467_Sentiment_Analysis_and_Visualization_of_Social_Media_Data_The_BostonMarathon_Bombings_test_case
- Schneider, K. (2003). *A Comparison of Event Models for Naïve Bayes Anti-Spam Email Filtering*. Retrieved from dl.acm.org: <https://dl.acm.org/citation.cfm?id=1067848>
- Vieweg, S. (2012). Retrieved from Situational Awareness in Mass Emergency: A behavioral and Linguistic Analysis of Microblogged Communications: <http://works.bepress.com/vieweg/15/>
- Wang, W., Wang, H., Dai, G., & Wang, H. (2006). *Visualization of Large Hierarchical Data by Circle Packing*. Retrieved from dl.acm.org: <https://dl.acm.org/citation.cfm?id=1124851>
- Weisstein, E. (n.d). *Circle Packing*. Retrieved from mathworld.wolfram.com: <http://mathworld.wolfram.com/circlepacking.html>
- Williams, E. (2010). *The Evolving Ecosystem*. Retrieved from Twitter.com: <http://blog.twitter.com/2010/09/evolving-ecosystem.html>
- Witten, I., & Frank, E. (2005). *Data Mining: Practical Machine Learning Tools and Techniques*. Morgan Kaufman Publishers.

APPENDICES

Appendix A – Imported python libraries

```
#!/usr/bin/env python -W ignore::DeprecationWarning
import csv
import itertools
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import warnings
from itertools import cycle
from sklearn.decomposition import LatentDirichletAllocation
from sklearn.feature_extraction import DictVectorizer
from sklearn.feature_extraction.text import CountVectorizer, TfidfTransformer
from sklearn.metrics import * #classification_report, confusion_matrix
from sklearn.model_selection import train_test_split
from sklearn.multiclass import OneVsRestClassifier
from sklearn.naive_bayes import MultinomialNB, BernoulliNB
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import label_binarize

tlen_sw = [x.strip('\n') for x in open("tlen_stopwords.txt", "r")]
warnings.simplefilter(action='ignore', category=FutureWarning)
```

Appendix B – Script to undersample 6-class dataset

```
df1 = pd.read_csv("hasNoClassmention_dataset.csv")

caa = df1[df1['defined_class']=='caution and advice']
cad = df1[df1['defined_class']=='casualties and damage']
domgs = df1[df1['defined_class']=='donations of money goods services']
pmfs = df1[df1['defined_class']=='people missing found seen']
infs = df1[df1['defined_class']=='information source']
notc = df1[df1['defined_class']=='none of the choices']
notc_count, infs_count, caa_count, cad_count, domgs_count, pmfs_count = df1.defined_class.value_counts()
min_sample = min([notc_count, caa_count, cad_count, pmfs_count, domgs_count])
notc_under, infs_under, caa_under, cad_under, domgs_under = notc.sample(min_sample), infs.sample(min_sample),
                                                                caa.sample(min_sample), cad.sample(min_sample),
                                                                domgs.sample(min_sample)

df_under = pd.concat([notc_under, infs_under, caa_under, cad_under, domgs_under, pmfs], ignore_index=True)
```

Appendix C – Script to data split for the 2-class and 6-class dataset

```
x, y = df1.tweet_text, label_binarize(df1.isInformative, classes=["notinformative", "informative"])
x_train, x_test, y_train, y_test = train_test_split(x, y, stratify=y, train_size=0.75, random_state=10)

x, y = df_under.tweet_text, df_under.defined_class
x_train, x_test, y_train, y_test = train_test_split(x, y, train_size=0.75, random_state=10)
```

Appendix D – Graphical confusion matrix plotter script

```
def plot_confusion_matrix(cm, classes,
                          normalize=False,
                          title='Confusion matrix',
                          cmap=plt.cm.Blues):

    if normalize:
        cm = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]
        print("Normalized confusion matrix")
    else:
        print('Confusion matrix, without normalization')

    print(cm)

    plt.imshow(cm, interpolation='nearest', cmap=cmap)
    plt.title(title)
    plt.colorbar()
    tick_marks = np.arange(len(classes))
    plt.xticks(tick_marks, classes, rotation=45)
    plt.yticks(tick_marks, classes)

    fmt = '.2f' if normalize else 'd'
    thresh = cm.max() / 2.
    for i, j in itertools.product(range(cm.shape[0]), range(cm.shape[1])):
        plt.text(j, i, format(cm[i, j], fmt),
                 horizontalalignment="center",
                 color="white" if cm[i, j] > thresh else "black")

    plt.ylabel('True label')
    plt.xlabel('Predicted label')
    plt.tight_layout()
```

Appendix E – Script of the BNB classifier, evaluator and confusion matrix plotter for the 6-Class dataset

```
i, j = 1, 4
for j in range(4,7):
    for i in range(1,4):
        text_clf = Pipeline([('vect', CountVectorizer(ngram_range=(i,j), stop_words='tl_enSW')),
                              ('tfidf', TfidfTransformer()),
                              ('clf', BernoulliNB()),])
        text_clf.fit(x_train,y_train)
        y_pred = text_clf.predict(x_test) #y_pred predicted labels based on mnb classifier
        print("=====|ngram_range(%s,%s)|===== "%(i,j))
        print("accuracy score %s"%accuracy_score(y_test,y_pred))
        print(classification_report(y_test, y_pred))
        cnf_matrix = confusion_matrix(y_test, y_pred)
        plt.figure()
        plot_confusion_matrix(cnf_matrix,classes = ['cad','caa','domgs','infs','notc','pmfs'],
                              title="confusion matrix")

        plt.show()
```

Appendix F – Script of the MNB classifier, evaluator and confusion matrix plotter for 6-Class dataset script

```
i, j = 1, 4
for j in range(4,7):
    for i in range(1,4):
        text_clf = Pipeline([('vect', CountVectorizer(ngram_range=(i,j),
                                                         stop_words=tl_enSW)),
                              ('tfidf', TfidfTransformer()),
                              ('clf', MultinomialNB()),])
        text_clf.fit(x_train,y_train)
        y_pred = text_clf.predict(x_test) #y_pred predicted labels based on mnb classifier
        print("=====ngram_range(%s,%s)===== "%(i,j))
        print("accuracy score %s"%accuracy_score(y_test,y_pred))
        print(classification_report(y_test, y_pred))
        cnf_matrix = confusion_matrix(y_test, y_pred)
        plt.figure()
        plot_confusion_matrix(cnf_matrix,classes = ['cad','caa',
                                                    'domgs','infs',
                                                    'notc','pmfs'], title="confusion matrix")

        plt.show()
```

Appendix G – Script of the BNB classifier, evaluator and confusion matrix plotter for the 2-Class dataset

```
i, j = 1, 4
for j in range(4,7):
    for i in range(1,4):
        text_clf = Pipeline([('vect', CountVectorizer(ngram_range=(i,j), stop_words=tl_enSW)),
                              ('tfidf', TfidfTransformer()),
                              ('clf', BernoulliNB()),])
        text_clf.fit(x_train,y_train)
        y_pred = text_clf.predict(x_test)
        print("====n-gram range(%s,%s)===="%(i,j))
        print("accuracy score %s"%accuracy_score(y_test,y_pred))
        print(classification_report(y_test, y_pred))
        cnf_matrix = confusion_matrix(y_test, y_pred)
        plt.figure()
        plot_confusion_matrix(cnf_matrix,classes = ["notinformative", "informative"],
                              title="BNB confusion matrix")

        plt.show()
        plt.savefig("BNB-binary_CM-%s-%s-ngrams.png"%(i,j))
```

Appendix H - Script of the MNB classifier, evaluator and confusion matrix plotter for the 2-Class dataset

```
i, j = 1, 6
for j in range(4,7):
    for i in range(1,4):
        text_clf = Pipeline([('vect', CountVectorizer(ngram_range=(i,j),
                                                         stop_words=tl_enSW)),
                              ('tfidf', TfidfTransformer()),
                              ('clf', MultinomialNB()),])
        text_clf.fit(x_train,y_train)
        y_pred = text_clf.predict(x_test)
        print("====n-gram range(%s,%s)===="%(i,j))
        print("accuracy score %s"%accuracy_score(y_test,y_pred))
        print(classification_report(y_test, y_pred))
        cnf_matrix = confusion_matrix(y_test, y_pred)
        plt.figure()
        plot_confusion_matrix(cnf_matrix,classes = ["notinformative", "informative"],
                              title="MNB confusion matrix")
        plt.show()
```

Appendix I – Script of the AUC-ROC curve and score comparison between MNB classifiers with n-gram (1,4) and (1,6) features

```
y_score_6 = text_clf.fit(x_train,y_train).predict_proba(x_test) #score for n_gram 1,6
text_clf2 = Pipeline([('vect', CountVectorizer(ngram_range=(1,4), stop_words=tl_enSW)),
                      ('tfidf', TfidfTransformer()),
                      ('clf', BernoulliNB()),])

y_score_4 = text_clf2.fit(x_train,y_train).predict_proba(x_test) #score for n_gram 1,4
fpr_6, tpr_6, thresh_6 = roc_curve(y_test, y_score_6[:,1])
fpr_4, tpr_4, thresh_4 = roc_curve(y_test, y_score_4[:,1])
roc_auc_6 = auc(fpr_6,tpr_6)
roc_auc_4 = auc(fpr_4,tpr_4)
plt.figure(figsize = (10,10))
plt.title('MNB ngramrange 1,4 and 1,6 AUROC')
plt.plot(fpr_6, tpr_6, c="blue", label=('AUC-1,6-binarydataset= %0.2f'%roc_auc_6))
plt.plot(fpr_4, tpr_4, c="red", label=('AUC-1,4-binarydataset= %0.2f'%roc_auc_4))
plt.legend(loc='lower right', prop={'size':8})
plt.plot([0,1],[0,1], color='lightgrey', linestyle='--')
plt.xlim([-0.05,1.0])
plt.ylim([0.0,1.05])
plt.ylabel('True Positive Rate')
plt.xlabel('False Positive Rate')
plt.show()
plt.savefig("MNB-binary(1,4)and(1,6)_ROC-AUC.png")
```

Appendix J – Script to export predicted results for 2-Class dataset

```
### save x-test and y-pred (predict_class) into new df
mnb_df_bin = pd.DataFrame({'tweet_text': x_test.tolist(),
                           'y_pred': y_pred.tolist(),
                           'y_test': y_test.tolist()})
mnb_df = pd.DataFrame({'tweet_text': x_test.tolist()})
mnb_df['predicted_inf_class'] = mnb_df['labelled_inf_class'] = ""
cnt1 = cnt0 = 0
for i, x in mnb_df_bin.iterrows():
    if x['y_pred'] == 1:
        mnb_df.at[i, 'predicted_inf_class'] = "informative"
    if x['y_pred'] == 0:
        mnb_df.at[i, 'predicted_inf_class'] = "notinformative"

for i, x in mnb_df_bin.iterrows():
    if x['y_test'] == [1]:
        mnb_df.at[i, 'labelled_inf_class'] = "informative"
    if x['y_test'] == [0]:
        mnb_df.at[i, 'labelled_inf_class'] = "notinformative"

new_df = df1.merge(mnb_df, how="inner", left_on="tweet_text", right_on="tweet_text")

### drop duplicates/discrepancies, save to new df
new_cleaned_df = new_df.drop_duplicates(subset=['tweet_text'])

### agreed_inf_class (must have similar classes bet.
### predicted_inf_class and labelled_inf_class, else discarded)
new_cleaned_df['agreed_inf_class'] = ""
count = 0

for i, x in new_cleaned_df.iterrows():
    if new_cleaned_df.at[i, 'predicted_inf_class'] == new_cleaned_df.at[i, 'labelled_inf_class']:
        new_cleaned_df.at[i, 'agreed_inf_class'] = new_cleaned_df.at[i, 'predicted_inf_class']
    else:
        new_cleaned_df = new_cleaned_df.drop([i])

### save result/classified data into
### new csv
new_cleaned_df.to_csv("ready_for_visualization.csv")
```

Appendix K – Script to export predicted results for 6-Class dataset

```
#### save x-test and y-pred into new df (for 6-class dataset)

mnb_df_bin = pd.DataFrame({'tweet_text': x_test.tolist(),
                           'y_pred': y_pred.tolist(),
                           'y_test': y_test.tolist()})
mnb_df = pd.DataFrame({'tweet_text': x_test.tolist()})
mnb_df['predicted_inf_class'] = mnb_df['labelled_inf_class'] = ""
cnt1 = cnt0 = 0
for i, x in mnb_df_bin.iterrows():
    mnb_df.at[i, 'predicted_inf_class'] = x['y_pred']
    mnb_df.at[i, 'labelled_inf_class'] = x['y_test']

new_df = df1.merge(mnb_df, how="inner", left_on="tweet_text", right_on="tweet_text")
new_df.sort_values(by='tweet_text')

new_cleaned_df = new_df.drop_duplicates(subset=['tweet_text'])
new_cleaned_df['agreed_inf_class'] = ""
count = 0

for i, x in new_cleaned_df.iterrows():
    if new_cleaned_df.at[i, 'predicted_inf_class'] == new_cleaned_df.at[i, 'labelled_inf_class']:
        new_cleaned_df.at[i, 'agreed_inf_class'] = new_cleaned_df.at[i, 'predicted_inf_class']
    else:
        new_cleaned_df = new_cleaned_df.drop([i])

new_cleaned_df.to_csv("6classes_visualization.csv")
```

Appendix L – Script to generate n-gram columns (1-6) for 2-Class dataset

```
#ngram column generator (for binary class) n-gram range (1,6)

import csv, pandas as pd, numpy as np
from nltk import ngrams
df = pd.read_csv("ready_for_visualization.csv")

df['1_grams_notc'] = df['1_grams_caa'] = df['1_grams_cad'] = df['1_grams_domgs'] = df['1_grams_pmfs'] = df['1_grams_infs'] = ""
df['2_grams_notc'] = df['2_grams_caa'] = df['2_grams_cad'] = df['2_grams_domgs'] = df['2_grams_pmfs'] = df['2_grams_infs'] = ""
df['3_grams_notc'] = df['3_grams_caa'] = df['3_grams_cad'] = df['3_grams_domgs'] = df['3_grams_pmfs'] = df['3_grams_infs'] = ""
df['4_grams_notc'] = df['4_grams_caa'] = df['4_grams_cad'] = df['4_grams_domgs'] = df['4_grams_pmfs'] = df['4_grams_infs'] = ""
df['5_grams_notc'] = df['5_grams_caa'] = df['5_grams_cad'] = df['5_grams_domgs'] = df['5_grams_pmfs'] = df['5_grams_infs'] = ""
df['6_grams_notc'] = df['6_grams_caa'] = df['6_grams_cad'] = df['6_grams_domgs'] = df['6_grams_pmfs'] = df['6_grams_infs'] = ""

grams1 = grams2 = grams3 = grams4 = grams5 = grams6 = []
for i, row in df.iterrows():

    grams1 = ngrams(row['tweet_text'].split(), 1)
    grams2 = ngrams(row['tweet_text'].split(), 2)
    grams3 = ngrams(row['tweet_text'].split(), 3)
    grams4 = ngrams(row['tweet_text'].split(), 4)
    grams5 = ngrams(row['tweet_text'].split(), 5)
    grams6 = ngrams(row['tweet_text'].split(), 6)
    l_notc = l_caa = l_cad = l_domgs = l_pmfs = l_infs = []

    for grams in grams1:
        if row['defined_class'] == "none of the choices":
            l_notc.append(" ".join(grams))
            df.at[i, '1_grams_notc'] = ";".join(l_notc)
        else:
            if row['defined_class'] == "caution and advice":
                l_caa.append(" ".join(grams))
                df.at[i, '1_grams_caa'] = ";".join(l_caa)
            if row['defined_class'] == "casualties and damage":
                l_cad.append(" ".join(grams))
                df.at[i, '1_grams_cad'] = ";".join(l_cad)
            if row['defined_class'] == "donations of money goods services":
                l_domgs.append(" ".join(grams))
                df.at[i, '1_grams_domgs'] = ";".join(l_domgs)
            if row['defined_class'] == "people missing found seen":
                l_pmfs.append(" ".join(grams))
                df.at[i, '1_grams_pmfs'] = ";".join(l_pmfs)
            if row['defined_class'] == "information source":
                l_infs.append(" ".join(grams))
                df.at[i, '1_grams_infs'] = ";".join(l_infs)
    l_notc = l_caa = l_cad = l_domgs = l_pmfs = l_infs = []
    for grams in grams2:
        if row['defined_class'] == "none of the choices":
            l_notc.append(" ".join(grams))
            df.at[i, '2_grams_notc'] = ";".join(l_notc)
        else:
            if row['defined_class'] == "caution and advice":
                l_caa.append(" ".join(grams))
                df.at[i, '2_grams_caa'] = ";".join(l_caa)
            if row['defined_class'] == "casualties and damage":
                l_cad.append(" ".join(grams))
                df.at[i, '2_grams_cad'] = ";".join(l_cad)
            if row['defined_class'] == "donations of money goods services":
                l_domgs.append(" ".join(grams))
                df.at[i, '2_grams_domgs'] = ";".join(l_domgs)
            if row['defined_class'] == "people missing found seen":
                l_pmfs.append(" ".join(grams))
                df.at[i, '2_grams_pmfs'] = ";".join(l_pmfs)
            if row['defined_class'] == "information source":
                l_infs.append(" ".join(grams))
                df.at[i, '2_grams_infs'] = ";".join(l_infs)
    l_notc = l_caa = l_cad = l_domgs = l_pmfs = l_infs = []
```

```

for grams in grams3:
    if row['defined_class'] == "none of the choices":
        l_notc.append(" ".join(grams))
        df.at[i,'3_grams_notc'] = ";".join(l_notc)
    else:
        if row['defined_class'] == "caution and advice":
            l_caa.append(" ".join(grams))
            df.at[i,'3_grams_caa'] = ";".join(l_caa)
        if row['defined_class'] == "casualties and damage":
            l_cad.append(" ".join(grams))
            df.at[i,'3_grams_cad'] = ";".join(l_cad)
        if row['defined_class'] == "donations of money goods services":
            l_domgs.append(" ".join(grams))
            df.at[i,'3_grams_domgs'] = ";".join(l_domgs)
        if row['defined_class'] == "people missing found seen":
            l_pmfs.append(" ".join(grams))
            df.at[i,'3_grams_pmfs'] = ";".join(l_pmfs)
        if row['defined_class'] == "information source":
            l_infs.append(" ".join(grams))
            df.at[i,'3_grams_infs'] = ";".join(l_infs)
l_notc = l_caa = l_cad = l_domgs = l_pmfs = l_infs = []
for grams in grams4:
    if row['defined_class'] == "none of the choices":
        l_notc.append(" ".join(grams))
        df.at[i,'4_grams_notc'] = ";".join(l_notc)
    else:
        if row['defined_class'] == "caution and advice":
            l_caa.append(" ".join(grams))
            df.at[i,'4_grams_caa'] = ";".join(l_caa)
        if row['defined_class'] == "casualties and damage":
            l_cad.append(" ".join(grams))
            df.at[i,'4_grams_cad'] = ";".join(l_cad)
        if row['defined_class'] == "donations of money goods services":
            l_domgs.append(" ".join(grams))
            df.at[i,'4_grams_domgs'] = ";".join(l_domgs)
        if row['defined_class'] == "people missing found seen":
            l_pmfs.append(" ".join(grams))
            df.at[i,'4_grams_pmfs'] = ";".join(l_pmfs)
        if row['defined_class'] == "information source":
            l_infs.append(" ".join(grams))
            df.at[i,'4_grams_infs'] = ";".join(l_infs)
l_notc = l_caa = l_cad = l_domgs = l_pmfs = l_infs = []
for grams in grams5:
    if row['defined_class'] == "none of the choices":
        l_notc.append(" ".join(grams))
        df.at[i,'5_grams_notc'] = ";".join(l_notc)
    else:
        if row['defined_class'] == "caution and advice":
            l_caa.append(" ".join(grams))
            df.at[i,'5_grams_caa'] = ";".join(l_caa)
        if row['defined_class'] == "casualties and damage":
            l_cad.append(" ".join(grams))
            df.at[i,'5_grams_cad'] = ";".join(l_cad)
        if row['defined_class'] == "donations of money goods services":
            l_domgs.append(" ".join(grams))
            df.at[i,'5_grams_domgs'] = ";".join(l_domgs)
        if row['defined_class'] == "people missing found seen":
            l_pmfs.append(" ".join(grams))
            df.at[i,'5_grams_pmfs'] = ";".join(l_pmfs)
        if row['defined_class'] == "information source":
            l_infs.append(" ".join(grams))
            df.at[i,'5_grams_infs'] = ";".join(l_infs)
l_notc = l_caa = l_cad = l_domgs = l_pmfs = l_infs = []

```

```

for grams in grams6:
    if row['defined_class'] == "none of the choices":
        l_notc.append(" ".join(grams))
        df.at[i,'6_grams_notc'] = ";".join(l_notc)
    else:
        if row['defined_class'] == "caution and advice":
            l_caa.append(" ".join(grams))
            df.at[i,'6_grams_caa'] = ";".join(l_caa)
        if row['defined_class'] == "casualties and damage":
            l_cad.append(" ".join(grams))
            df.at[i,'6_grams_cad'] = ";".join(l_cad)
        if row['defined_class'] == "donations of money goods services":
            l_domgs.append(" ".join(grams))
            df.at[i,'6_grams_domgs'] = ";".join(l_domgs)
        if row['defined_class'] == "people missing found seen":
            l_pmfs.append(" ".join(grams))
            df.at[i,'6_grams_pmfs'] = ";".join(l_pmfs)
        if row['defined_class'] == "information source":
            l_infs.append(" ".join(grams))
            df.at[i,'6_grams_infs'] = ";".join(l_infs)
l_notc = l_caa = l_cad = l_domgs = l_pmfs = l_infs = []
grams1 = grams2 = grams3 = grams4 = grams5 = grams6 = []
df.to_csv("ready_for_visualization.csv", index=None)

```

Appendix M - Script to generate n-gram columns (1-4) for 6-Class dataset

```
#ngram column generator (for 6 class) n,gram range (1,4)

import csv, pandas as pd, numpy as np
from nltk import ngrams
df = pd.read_csv("6classes_visualization.csv")

df['1_grams_notc']=df['1_grams_caa']=df['1_grams_cad']=df['1_grams_domgs']=df['1_grams_pmfs']=df['1_grams_infs']=" "
df['2_grams_notc']=df['2_grams_caa']=df['2_grams_cad']=df['2_grams_domgs']=df['2_grams_pmfs']=df['2_grams_infs']=" "
df['3_grams_notc']=df['3_grams_caa']=df['3_grams_cad']=df['3_grams_domgs']=df['3_grams_pmfs']=df['3_grams_infs']=" "
df['4_grams_notc']=df['4_grams_caa']=df['4_grams_cad']=df['4_grams_domgs']=df['4_grams_pmfs']=df['4_grams_infs']=" "
grams1 = grams2 = grams3 = grams4 = []
for i, row in df.iterrows():

    grams1 = ngrams(row['tweet_text'].split(),1)
    grams2 = ngrams(row['tweet_text'].split(),2)
    grams3 = ngrams(row['tweet_text'].split(),3)
    grams4 = ngrams(row['tweet_text'].split(),4)
    l_notc = l_caa = l_cad = l_domgs = l_pmfs = l_infs = []

    for grams in grams1:
        if row['agreed_inf_class'] == "none of the choices":
            l_notc.append(" ".join(grams))
            df.at[i,'1_grams_notc'] = ";".join(l_notc)
        else:
            if row['agreed_inf_class'] == "caution and advice":
                l_caa.append(" ".join(grams))
                df.at[i,'1_grams_caa'] = ";".join(l_caa)
            if row['agreed_inf_class'] == "casualties and damage":
                l_cad.append(" ".join(grams))
                df.at[i,'1_grams_cad'] = ";".join(l_cad)
            if row['agreed_inf_class'] == "donations of money goods services":
                l_domgs.append(" ".join(grams))
                df.at[i,'1_grams_domgs'] = ";".join(l_domgs)
            if row['agreed_inf_class'] == "people missing found seen":
                l_pmfs.append(" ".join(grams))
                df.at[i,'1_grams_pmfs'] = ";".join(l_pmfs)
            if row['agreed_inf_class'] == "information source":
                l_infs.append(" ".join(grams))
                df.at[i,'1_grams_infs'] = ";".join(l_infs)
    l_notc = l_caa = l_cad = l_domgs = l_pmfs = l_infs = []
    for grams in grams2:
        if row['agreed_inf_class'] == "none of the choices":
            l_notc.append(" ".join(grams))
            df.at[i,'2_grams_notc'] = ";".join(l_notc)
        else:
            if row['agreed_inf_class'] == "caution and advice":
                l_caa.append(" ".join(grams))
                df.at[i,'2_grams_caa'] = ";".join(l_caa)
            if row['agreed_inf_class'] == "casualties and damage":
                l_cad.append(" ".join(grams))
                df.at[i,'2_grams_cad'] = ";".join(l_cad)
            if row['agreed_inf_class'] == "donations of money goods services":
                l_domgs.append(" ".join(grams))
                df.at[i,'2_grams_domgs'] = ";".join(l_domgs)
            if row['agreed_inf_class'] == "people missing found seen":
                l_pmfs.append(" ".join(grams))
                df.at[i,'2_grams_pmfs'] = ";".join(l_pmfs)
            if row['agreed_inf_class'] == "information source":
                l_infs.append(" ".join(grams))
                df.at[i,'2_grams_infs'] = ";".join(l_infs)
    l_notc = l_caa = l_cad = l_domgs = l_pmfs = l_infs = []
```

```

for grams in grams3:
    if row['agreed_inf_class'] == "none of the choices":
        l_notc.append(" ".join(grams))
        df.at[i, '3_grams_notc'] = ";" + ".join(l_notc)
    else:
        if row['agreed_inf_class'] == "caution and advice":
            l_caa.append(" ".join(grams))
            df.at[i, '3_grams_caa'] = ";" + ".join(l_caa)
        if row['agreed_inf_class'] == "casualties and damage":
            l_cad.append(" ".join(grams))
            df.at[i, '3_grams_cad'] = ";" + ".join(l_cad)
        if row['agreed_inf_class'] == "donations of money goods services":
            l_domgs.append(" ".join(grams))
            df.at[i, '3_grams_domgs'] = ";" + ".join(l_domgs)
        if row['agreed_inf_class'] == "people missing found seen":
            l_pmfs.append(" ".join(grams))
            df.at[i, '3_grams_pmfs'] = ";" + ".join(l_pmfs)
        if row['agreed_inf_class'] == "information source":
            l_infs.append(" ".join(grams))
            df.at[i, '3_grams_infs'] = ";" + ".join(l_infs)
l_notc = l_caa = l_cad = l_domgs = l_pmfs = l_infs = []
for grams in grams4:
    if row['agreed_inf_class'] == "none of the choices":
        l_notc.append(" ".join(grams))
        df.at[i, '4_grams_notc'] = ";" + ".join(l_notc)
    else:
        if row['agreed_inf_class'] == "caution and advice":
            l_caa.append(" ".join(grams))
            df.at[i, '4_grams_caa'] = ";" + ".join(l_caa)
        if row['agreed_inf_class'] == "casualties and damage":
            l_cad.append(" ".join(grams))
            df.at[i, '4_grams_cad'] = ";" + ".join(l_cad)
        if row['agreed_inf_class'] == "donations of money goods services":
            l_domgs.append(" ".join(grams))
            df.at[i, '4_grams_domgs'] = ";" + ".join(l_domgs)
        if row['agreed_inf_class'] == "people missing found seen":
            l_pmfs.append(" ".join(grams))
            df.at[i, '4_grams_pmfs'] = ";" + ".join(l_pmfs)
        if row['agreed_inf_class'] == "information source":
            l_infs.append(" ".join(grams))
            df.at[i, '4_grams_infs'] = ";" + ".join(l_infs)
l_notc = l_caa = l_cad = l_domgs = l_pmfs = l_infs = []
grams1 = grams2 = grams3 = grams4 = []
df.to_csv("6classes visualization.csv", index=None)

```

Appendix N – Python Django’s settings.py

```
import os,netifaces as ni

BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
SECRET_KEY = '24%hrf$24slduv@d=12@k%2=*@ipm^lxls2$av9-@m2ewtvp76'
DEBUG = True
curr_ip = ni.ifaddresses('enp0s3')[ni.AF_INET][0]['addr']
ALLOWED_HOSTS = [curr_ip, 'localhost']
INSTALLED_APPS = ['django.contrib.admin', 'django.contrib.auth', 'django.contrib.contenttypes',
                  'django.contrib.sessions', 'django.contrib.messages', 'django.contrib.staticfiles',
                  'crispy_forms', 'Person', 'datasci',
                  ]
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'thesis.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [
            os.path.join(BASE_DIR, 'templates')
        ],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    ],
]

WSGI_APPLICATION = 'thesis.wsgi.application'

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': os.path.join(BASE_DIR, 'db.sqlite3'),
    }
}

AUTH_PASSWORD_VALIDATORS = [
    { 'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator', },
    { 'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator', },
    { 'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator', },
    { 'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator', },
]

LANGUAGE_CODE = 'en-us'

TIME_ZONE = 'Asia/Manila'

USE_I18N = True

USE_L10N = True

USE_TZ = True

STATIC_URL = '/static/'
STATICFILES_DIRS = [
    os.path.join(BASE_DIR, "static"),
]
CRISPY_TEMPLATE_PACK = 'bootstrap4'

MEDIA_ROOT = os.path.join(BASE_DIR, 'media')
MEDIA_URL = '/media/'
```

Appendix O – Python Django's urls.py

```
from thesis import settings
from django.contrib import admin
from django.contrib.staticfiles.urls import static
from django.urls import path
from Person.views import PersonListView, PersonCreateView, PersonUpdateView
import datsci.views
import datsci.views2
#import datsci.views3
urlpatterns = [
    path('', datsci.views.circle_pack_view),
    path('datsci/circle/', datsci.views.circle_pack_view),
    path('datsci/circle_7/', datsci.views2.circle_pack_view),
    path('datsci/circ_flat/', datsci.views.circle_flat_view),
    path('datsci/tester/', datsci.views.single_tweet_test),
]

urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

Appendix P – Python Django's forms.py

```
from django import forms

class TweetForm(forms.Form):
    the_tweet = forms.CharField(label="Tweet Text",
                                min_length=20,
                                max_length=280,
                                required=True,
                                widget = forms.TextInput(attrs={'class':'form-control',
                                                                'placeholder':'Enter tweet'}))
```

Appendix Q – Python Django’s views.py

```
#!/usr/bin/env python -W ignore::DeprecationWarning
import csv
import itertools
import json
import matplotlib.pyplot as plt
import numpy as np
import os
import PIL, PIL.Image
import pandas as pd
import random
import warnings
from .forms import TweetForm
from collections import Counter
from django.http import HttpResponse
from django.shortcuts import render
from itertools import cycle
from sklearn.decomposition import LatentDirichletAllocation
from sklearn.feature_extraction import DictVectorizer
from sklearn.feature_extraction.text import CountVectorizer, TfidfTransformer
from sklearn.metrics import * #classification_report, confusion_matrix
from sklearn.model_selection import train_test_split
from sklearn.multiclass import OneVsRestClassifier
from sklearn.naive_bayes import MultinomialNB, BernoulliNB
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import label_binarize
from string import Template

base_dir = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
CLASSES = ["notinformative", "informative"]
tl_enSW = [x.strip('\n') for x in open(os.path.join(base_dir, "datasci/tlen_stopwords.txt"), "r")]
warnings.simplefilter(action='ignore', category=FutureWarning)
df1 = pd.read_csv(os.path.join(base_dir, "datasci/ready_for_visualization.csv"))
df2 = pd.read_csv(os.path.join(base_dir, "datasci/6classes_visualization.csv"))

### output.json node generator
def parse_g_child(col, df_num):
    if df_num == 1:
        df = df1
    if df_num == 2:
        df = df2
    bigtxt = df[col].dropna()
    bigg = ""
    for x in bigtxt:
        bigg += str(x)
    wrds = bigg.split(";")
    wrds = Counter(wrds).most_common(10)
    resp = {'name' : col, "children" : [], "size":""}
    for wrd in wrds:
        doc = {'name' : wrd[0], "children": [], "size": wrd[1]}
        resp['children'].append(doc)
    return resp
```

```

def parse_child_2c(col,df_num):
    if df_num == 1:
        df = df1
    if df_num == 2:
        df = df2
    bigtxt = ";".join(df[col].dropna())
    wrds = bigtxt.split(";")
    wrds = Counter(wrds).most_common()
    notc = caa = cad = domgs = pmfs = infs = {}
    for wrd in wrds:
        if wrd[0] == "none of the choices":
            notc = {'name' : wrd[0], "children": []}
            for x in ("1_grams_notc","2_grams_notc","3_grams_notc","4_grams_notc","5_grams_notc","6_grams_notc"):
                child = parse_g_child(x,df_num)
                notc['children'].append(child)
        else:
            if wrd[0] == "caution and advice":
                caa = {'name' : wrd[0], "children": []}
                for x in ("1_grams_caa","2_grams_caa","3_grams_caa","4_grams_caa","5_grams_caa","6_grams_caa"):
                    child = parse_g_child(x,df_num)
                    caa['children'].append(child)
            if wrd[0] == "casualties and damage":
                cad = {'name' : wrd[0], "children": []}
                for x in ("1_grams_cad","2_grams_cad","3_grams_cad","4_grams_cad","5_grams_cad","6_grams_cad"):
                    child = parse_g_child(x,df_num)
                    cad['children'].append(child)
            if wrd[0] == "donations of money goods services":
                domgs = {'name' : wrd[0], "children": []}
                for x in ("1_grams_domgs","2_grams_domgs","3_grams_domgs",
                        "4_grams_domgs","5_grams_domgs","6_grams_domgs"):
                    child = parse_g_child(x,df_num)
                    domgs['children'].append(child)
            if wrd[0] == "people missing found seen":
                pmfs = {'name' : wrd[0], "children": []}
                for x in ("1_grams_pmfs","2_grams_pmfs","3_grams_pmfs","4_grams_pmfs","5_grams_pmfs","6_grams_pmfs"):
                    child = parse_g_child(x,df_num)
                    pmfs['children'].append(child)
            if wrd[0] == "information source":
                infs = {'name' : wrd[0], "children": []}
                for x in ("1_grams_infs","2_grams_infs","3_grams_infs","4_grams_infs","5_grams_infs","6_grams_infs"):
                    child = parse_g_child(x,df_num)
                    infs['children'].append(child)
    return notc,caa,cad,domgs,pmfs,infs

def parse_child_2c(col,df_num):
    if df_num == 1:
        df = df1
    if df_num == 2:
        df = df2
    bigtxt = ";".join(df[col].dropna())
    wrds = bigtxt.split(";")
    wrds = Counter(wrds).most_common()
    notc = caa = cad = domgs = pmfs = infs = {}
    for wrd in wrds:
        if wrd[0] == "none of the choices":
            notc = {'name' : wrd[0], "children": []}
            for x in ("1_grams_notc","2_grams_notc","3_grams_notc","4_grams_notc","5_grams_notc","6_grams_notc"):
                child = parse_g_child(x,df_num)
                notc['children'].append(child)

```

```

else:
    if wrd[0] == "caution and advice":
        caa = {'name' : wrd[0], "children": []}
        for x in ("1_grams_caa", "2_grams_caa", "3_grams_caa", "4_grams_caa"):
            child = parse_g_child(x, df_num)
            caa['children'].append(child)
    if wrd[0] == "casualties and damage":
        cad = {'name' : wrd[0], "children": []}
        for x in ("1_grams_cad", "2_grams_cad", "3_grams_cad", "4_grams_cad"):
            child = parse_g_child(x, df_num)
            cad['children'].append(child)
    if wrd[0] == "donations of money goods services":
        domgs = {'name' : wrd[0], "children": []}
        for x in ("1_grams_domgs", "2_grams_domgs", "3_grams_domgs", "4_grams_domgs"):
            child = parse_g_child(x, df_num)
            domgs['children'].append(child)
    if wrd[0] == "people missing found seen":
        pmfs = {'name' : wrd[0], "children": []}
        for x in ("1_grams_pmfs", "2_grams_pmfs", "3_grams_pmfs", "4_grams_pmfs"):
            child = parse_g_child(x, df_num)
            pmfs['children'].append(child)
    if wrd[0] == "information source":
        infs = {'name' : wrd[0], "children": []}
        for x in ("1_grams_infs", "2_grams_infs", "3_grams_infs", "4_grams_infs"):
            child = parse_g_child(x, df_num)
            infs['children'].append(child)
    return notc, caa, cad, domgs, pmfs, infs
def parse(col, df_num):
    if df_num == 1:
        df = df1
    if df_num == 2:
        df = df2
    bigtxt = ";".join(df[col].dropna())
    wrds = bigtxt.split(";")
    wrds = Counter(wrds).most_common()
    for wrd in wrds:
        if wrd[0] == "informative":
            resp = {'name' : wrd[0], "children": []}
            if df_num == 1:
                notc, caa, cad, domgs, pmfs, infs = parse_child_2c("defined_class", df_num)
            if df_num == 2:
                notc, caa, cad, domgs, pmfs, infs = parse_child_6c("agreed_inf_class", df_num)
            resp['children'].append(caa)
            resp['children'].append(cad)
            resp['children'].append(domgs)
            resp['children'].append(pmfs)
            resp['children'].append(infs)
        else:
            resp2 = {'name' : wrd[0], "children": []}
            if df_num == 1:
                notc, caa, cad, domgs, pmfs, infs = parse_child_2c("defined_class", df_num)
            if df_num == 2:
                notc, caa, cad, domgs, pmfs, infs = parse_child_6c("agreed_inf_class", df_num)
            resp2['children'].append(notc)
    return resp, resp2

```

```

def circle_pack_view(request):
    results2 = {'name' : 'flare', "children" : [], "size":""}
    info,info2 = parse("isInformative",1)
    results2['children'].append(info)
    results2['children'].append(info2)
    json_path = "static/js/output.json"
    with open(os.path.join(base_dir,json_path), 'w') as outfile:
        json.dump(results2, outfile)
    context = {'output_file':"/"+json_path, 'd3lib':"static/js/d3.v4.min.js", 'pageno':1}
    return render(request,"circle_pack.html",context)

def circle_flat_view(request):
    results2 = {'name' : 'flare', "children" : [], "size":""}
    notc,caa,cad,domgs,pmfs,info = parse_child_6c("agreed_inf_class",2)
    results2['children'].append(notc)
    results2['children'].append(caa)
    results2['children'].append(cad)
    results2['children'].append(domgs)
    results2['children'].append(pmfs)
    results2['children'].append(info)
    json_path = "static/js/output2.json"
    with open(os.path.join(base_dir,json_path), 'w') as outfile:
        json.dump(results2, outfile)
    context = {'output_file':"/"+json_path, 'd3lib':"static/js/d3.v4.min.js", 'pageno':2}
    return render(request,"circle_pack.html",context)

def single_tweet_test(request):
    if request.method == 'POST':
        form = TweetForm(request.POST)
        if form.is_valid():
            df1 = pd.read_csv(os.path.join(base_dir,"datasci/hasNoClassmention_dataset.csv"))
            x, y = df1.tweet_text, label_binarize(df1.isInformative,classes=CLASSES)
            x_train, x_test, y_train, y_test = train_test_split(x, y, stratify=y, train_size=0.75, random_state=10)
            text_clf = Pipeline([('vect', CountVectorizer(ngram_range=(1,6), stop_words=tl_enSW)),
                                ('tfidf', TfidfTransformer()),
                                ('clf', MultinomialNB()),])
            text_clf.fit(x_train,y_train)

            tweet = form.cleaned_data['the_tweet']
            cust_pred = text_clf.predict(pd.Series(tweet))
            label = "informative" if cust_pred == [1] else "not informative"
            if len(tweet) <= 3:
                label = "not informative"
            context = {'form':form,'tweet':tweet,'predict': label, 'pageno':3}

        else:
            form = TweetForm()
            context = {'form': form, 'pageno':3}
    return render(request, 'tweet.html', context)

```

Appendix R – base.html the base template for all web pages

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">
    {% load static %}
    <link rel="icon" href="{% static 'icon/baseline_poll_black_48dp.png' %}" type="image/x-icon" sizes="16x16"/>
    <link rel="stylesheet" href="{% static 'css/bootstrap.css' %}" />
    <script src="{% static 'js/jquery.js' %}"></script>
    <script src="{% static 'js/bootstrap.js' %}"></script>
    <title>Tweet Tracker</title>
    <style>
      .node { cursor: pointer;}
      .node:hover { stroke: #000; stroke-width: 1.5px;}
      .node--leaf { fill: white; pointer-events: auto;}
      .label { color: red; font: 20px "Helvetica Neue", Helvetica, Arial, sans-serif; text-anchor: middle;
      text-shadow: 0 1px 0 #fff, 1px 0 0 #fff, -1px 0 0 #fff, 0 -1px 0 #fff;}
      .label, .node--root { pointer-events: none;}
    </style>
  </head>
  <body>
    <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
      <div class="container-fluid">
        <div class="navbar-header">
          <a class="navbar-brand" href="#">TweetTracker</a>
        </div>
        <ul class="navbar-nav mr-auto">
          <li class="nav-item {% if pageno == 1 %}active{% endif %}"><a class="nav-link" href="/datsci/circle/">
            2-Class Topics</a></li>
          <li class="nav-item {% if pageno == 2 %}active{% endif %}"><a class="nav-link" href="/datsci/circ_flat/">
            6-Class Topics</a></li>
          <li class="nav-item {% if pageno == 3 %}active{% endif %}"><a class="nav-link" href="/datsci/tester/">
            label-a-tweet</a></li>
        </ul>
      </div>
    </nav>
    <div class="container">
      <div class="row justify-content-center">
        <div class="col-8">
          <hr class="mt-0 mb-4">
          {% block content %}
          {% endblock %}
        </div>
      </div>
    </div>
  </body>
</html>
```

Appendix S – circle_pack.html, the template which implements the d3.js circle packing codes

```
{% extends 'base.html' %}
{% block content %}
<svg id="theframe" height="900" width="900" viewBox="0 0 900 900" preserveAspectRatio="xMidYMid meet"></svg>
<script src="{{ d3lib }}"></script>
<script>
var svg = d3.select("svg"),
    margin = 20,
    diameter = +svg.attr("width"),
    g = svg.append("g").attr("transform", "translate(" + diameter / 2 + "," + diameter / 2 + ")");

var color = d3.scaleLinear()
    .domain([-1, 5])
    .range(["hsl(152,80%,80%)", "hsl(228,30%,40%)"])
    .interpolate(d3.interpolateHcl);

var pack = d3.pack()
    .size([diameter - margin, diameter - margin])
    .padding(2);

d3.json("{{ output_file }}", function(error, root) {
    if (error) throw error;

    root = d3.hierarchy(root)
        .sum(function(d) { return d.size; })
        .sort(function(a, b) { return b.value - a.value; });

    var focus = root,
        nodes = pack(root).descendants(),
        view;

    var circle = g.selectAll("circle")
        .data(nodes)
        .enter().append("circle")
        .attr("class", function(d) { return d.parent ? d.children ? "node" : "node node--leaf" : "node node--root"; })
        .style("fill", function(d) { return d.children ? color(d.depth) : null; })
        .on("click", function(d) { if (focus !== d) zoom(d), d3.event.stopPropagation(); });
    circle.append("svg:title")
        .text(function(d) {
            return d.data.name + "\n(" + d.value + ")";
        });

    var text = g.selectAll("text")
        .data(nodes)
        .enter().append("text")
        .attr("class", "label")
        .style("fill-opacity", function(d) { return d.parent === root ? 1 : 0; })
        .style("display", function(d) { return d.parent === root ? "inline" : "none"; })
        .text(function(d) { return d.data.name; });

```

```

var node = g.selectAll("circle,text");

svg
  .style("background", color(-1))
  .on("click", function() { zoom(root); });

zoomTo([root.x, root.y, root.r * 2 + margin]);

function zoom(d) {
  var focus0 = focus; focus = d;

  var transition = d3.transition()
    .duration(d3.event.altKey ? 7500 : 750)
    .tween("zoom", function(d) {
      var i = d3.interpolateZoom(view, [focus.x, focus.y, focus.r * 2 + margin]);
      return function(t) { zoomTo(i(t)); };
    });

  transition.selectAll("text")
    .filter(function(d) { return d.parent === focus || this.style.display === "inline"; })
    .style("fill-opacity", function(d) { return d.parent === focus ? 1 : 0; })
    .on("start", function(d) { if (d.parent === focus) this.style.display = "inline"; })
    .on("end", function(d) { if (d.parent !== focus) this.style.display = "none"; });
}

function zoomTo(v) {
  var k = diameter / v[2]; view = v;
  node.attr("transform", function(d) { return "translate(" + (d.x - v[0]) * k + "," + (d.y - v[1]) * k + ")"; });
  circle.attr("r", function(d) { return d.r * k; });
}
});
</script>
{% endblock %}

```

Appendix T – List of Stop Words

a about above across after afterwards again against akin aking ako alin all almost alone along already also although always am amin aming among amongst amount an and ang ano another anumang any anyhow anyone anything anyway anywhere apat are arent around as at atin ating ay baba bababa back bago bakit bawat be became because become becomes becoming been before beforehand behind being below beside besides between beyond bilang bill both bottom but by call can cannot cant co con could couldnt cry dahil dalawa dapat de describe detail did didnt din dito diyan do does doesnt doing done dont doon down due during each eg eight either eleven else elsewhere empty enough etc even ever every everyone everything everywhere except few fifteen fifty fill find fire first five for former formerly forty found four from front full further gagawin galing ganyan gawin gayunman get ginagawa ginawa ginawang give go gumawa gusto habang had hadnt hanggang has hasnt have havent having he hed hell hence her here hereafter hereby herein heres hereupon hers herself hes higit him himself hindi his how however hows hundred huwag i iba ibaba ibabaw ibig id ie if ikaw ilagay ilalim ilan ilang ill im in inc indeed interest into inyong is isa isang isnt it itaas ito its itself ive iyan iyo iyon iyong ka kahit kailan kailangan kailanman kami kanila kanilang kanino kanya kanyang kapag kapwa karamihan katiyakan katulad kay kaya kaysa keep kesa ko kong kulang kumuha kung laban labas lahat lamang lang last latter latterly least less lets likod lima ltd maaari maaaring made maging mahusay makita many marami marapat masyado may mayroon me meanwhile mga might mill mine minsan mismo more moreover most mostly move much mula muli must mustnt my myself na nabanggit naging nagkaroon nais nakita name namely namin napaka narapat narito nasaan neither never nevertheless next ng ngaun ngayon ngayun ni nila nilang nine nito niya niyang no nobody none noon noone nor not nothing now nowhere o of off often on once one only onto or other others otherwise ought our ours ourselves out over own pa paano pababa paggawa pagiging pagitan pagka pagkakaroon pagkatapos palabas pamamagitan panahon pangalawa para paraan parehas pareho part pataas per perhaps pero please pumunta pumupunta put pwede rather re rin sa saan sabi sabihin saka same sarili see seem seemed seeming seems serious several shant she shed shell shes should shouldnt show side sila since sincere sino six sixty siya so some somehow someone something sometime sometimes somewhere still such system taas take tatlo tayo ten than that thats the their theirs them themselves then thence there thereafter thereby therefore therein theres thereupon these they theyd theyll theyre theyve thick thin third this those though three through throughout thru thus to together too top toward towards tulad tungkol twelve twenty two ulit un una under until up upon us very via walang was wasnt we wed well were werent weve what whatever whats when whence whenever whens where whereafter whereas whereby wherein wheres whereupon wherever whether which while whither who whoever whole whom whos whose why whys will with within without wont would wouldnt yet you youd youll your youre yours yourself yourselves youve